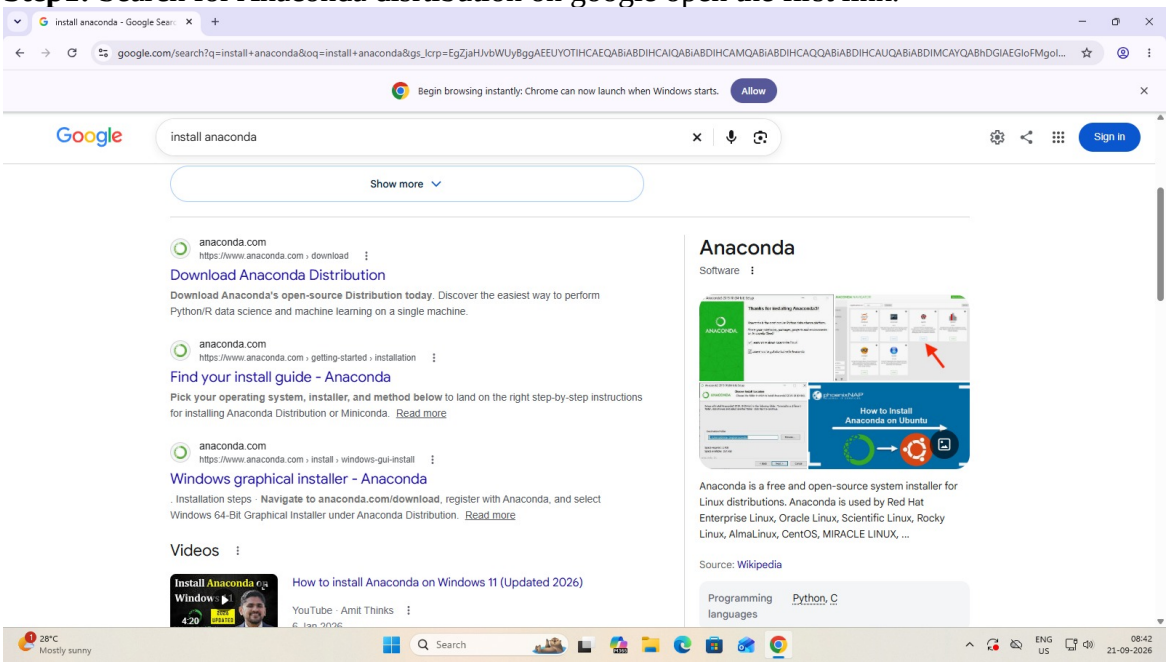
Practical 1

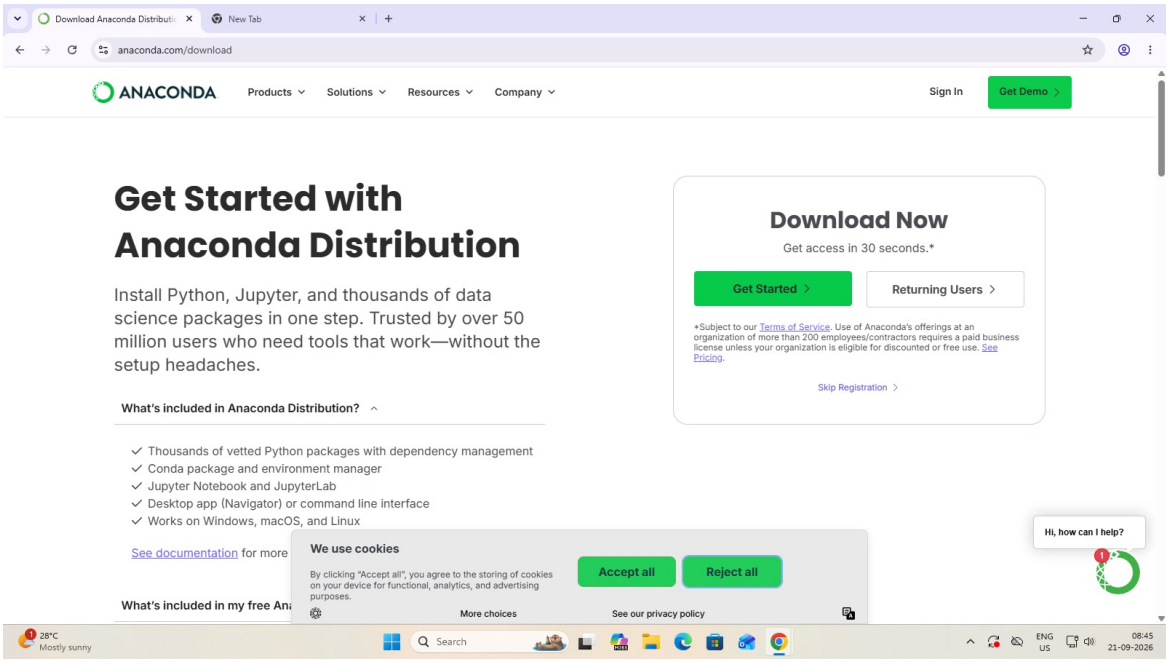
Aim: Installing and runing asimple python code in Anaconda

Steps:

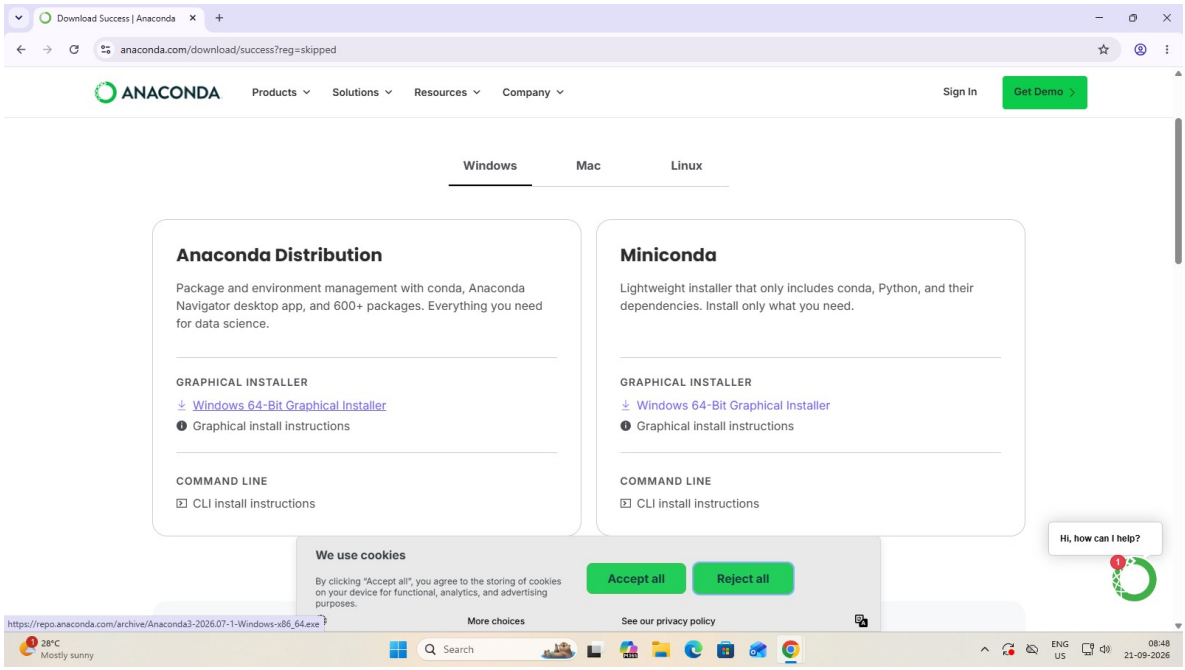
Step1: Search for Anaconda disrtibution on google open the first link.



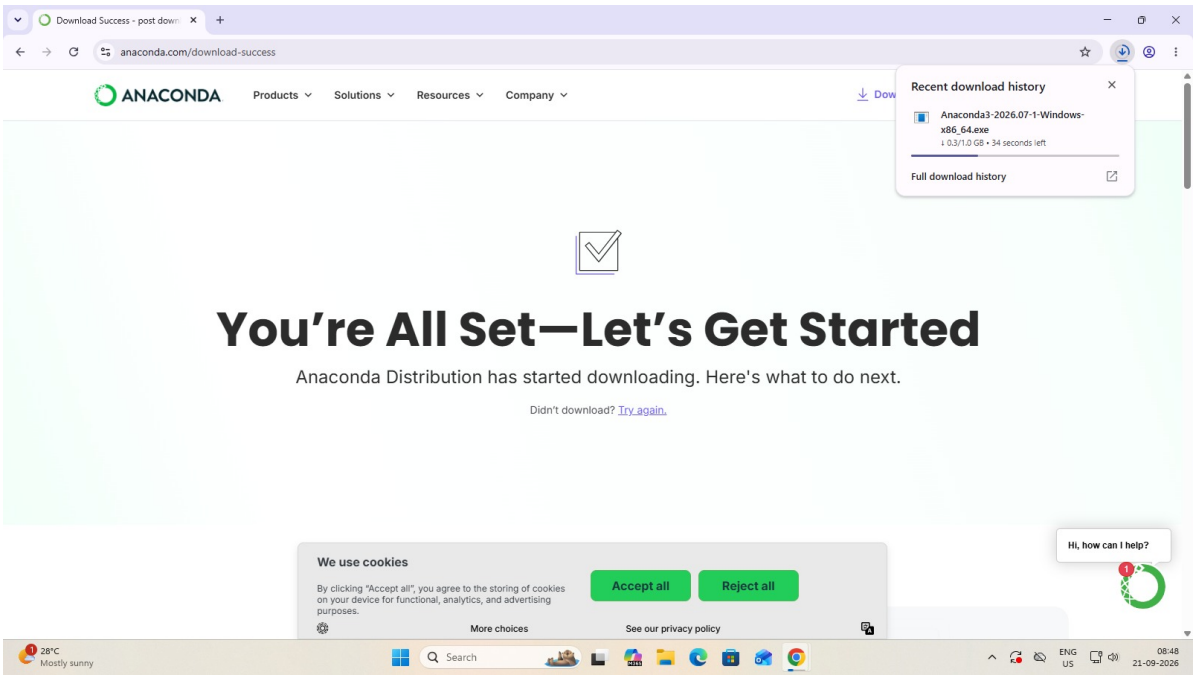
Step 2: Open the first link



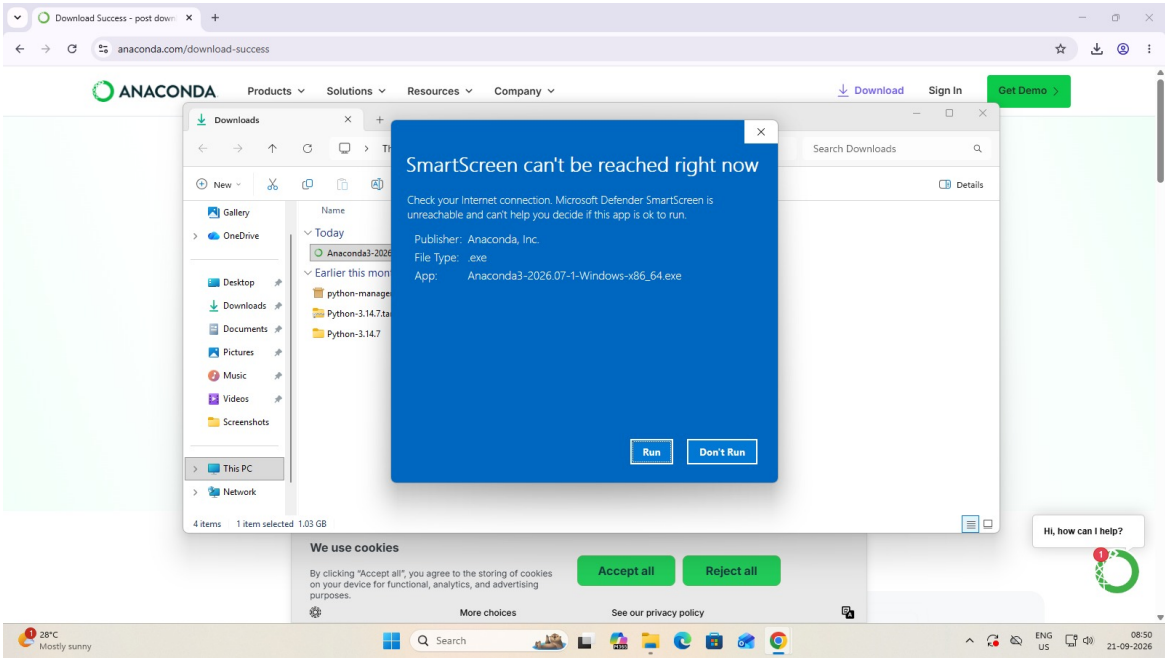
Step 3 Click on download and slect your Operating system



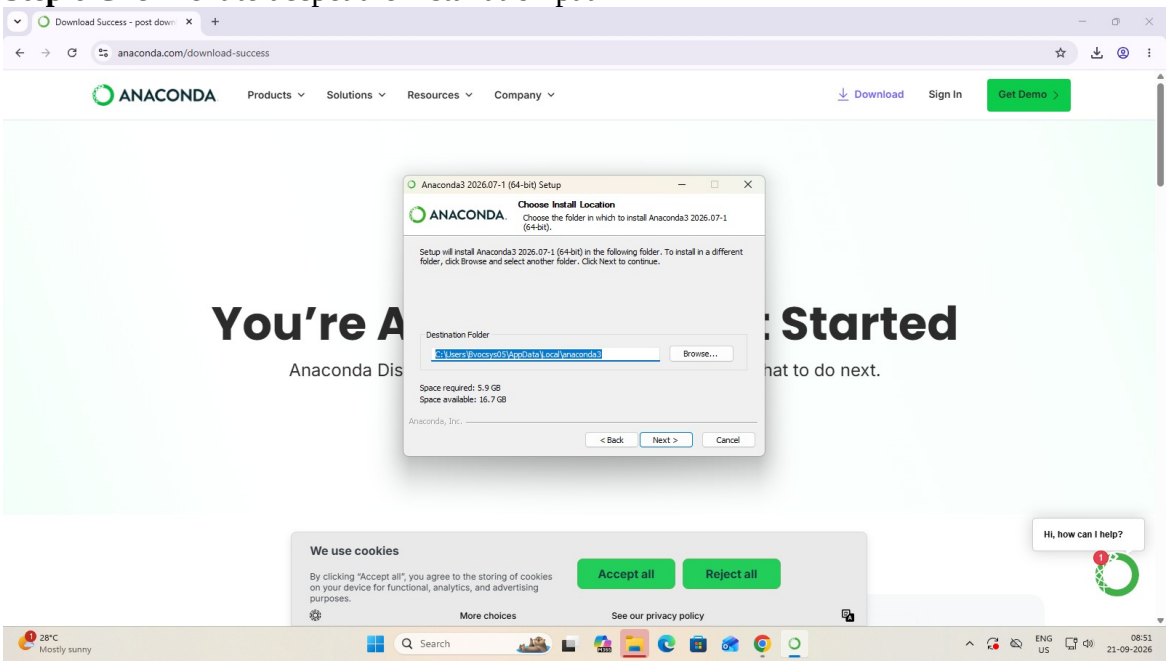
Step 4 Select on Graphical installer link then your Download should start



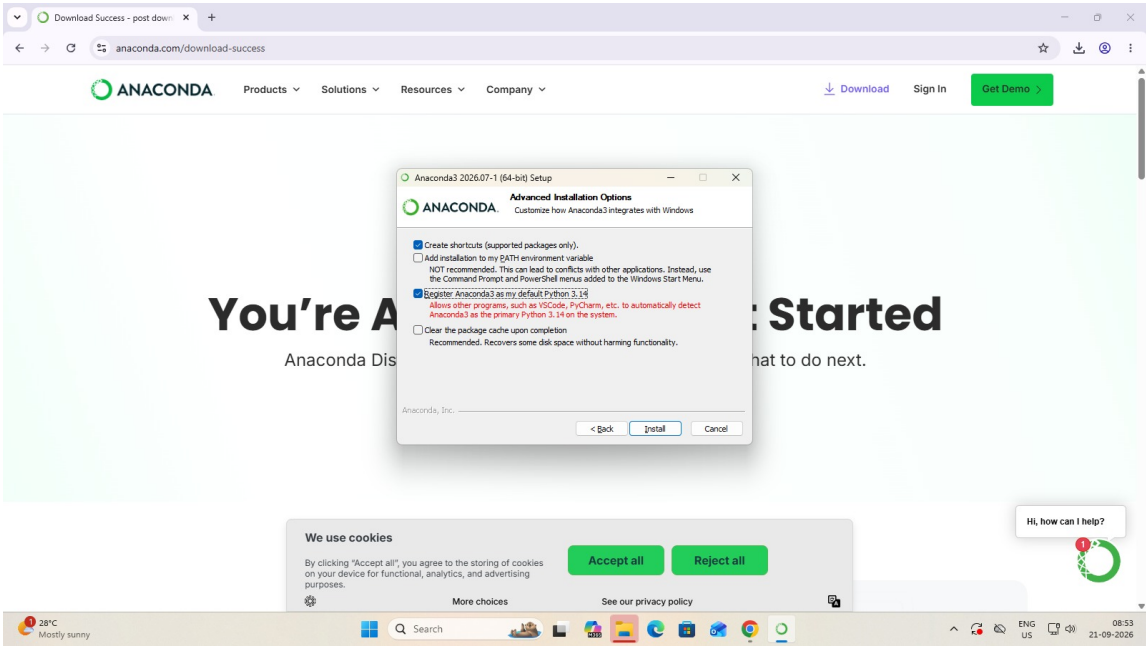
Step 5 Open the downloaded file. Then windows smart screen window will pop select Run



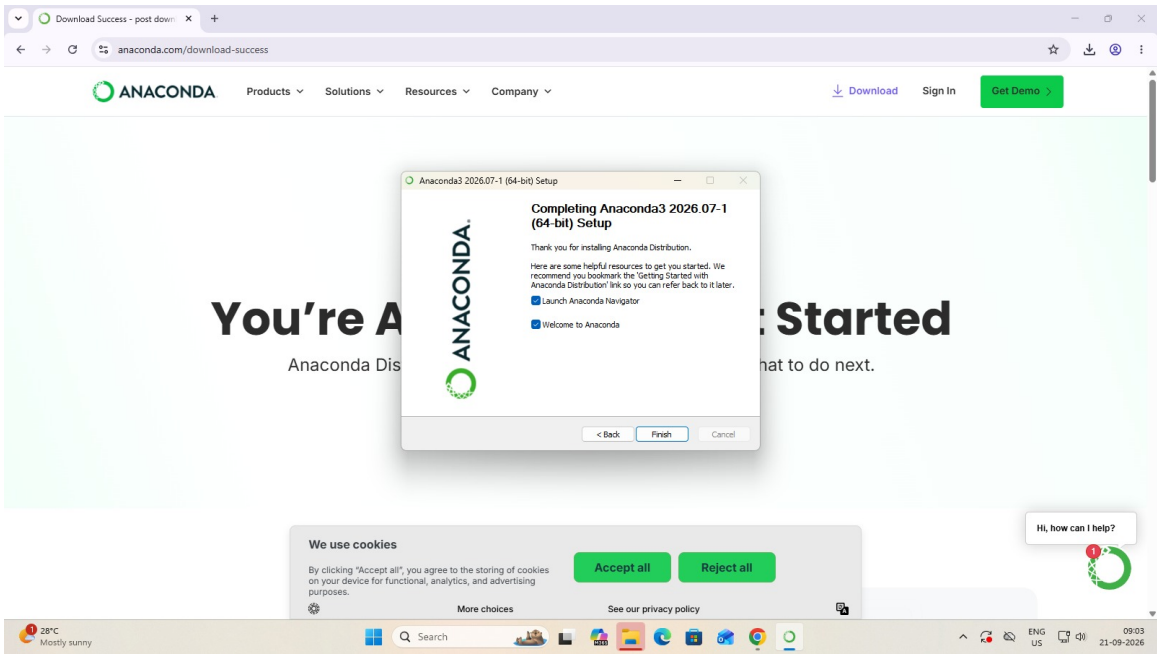
Step 6 Click next to accpet the installation path



Step 7 Select create shortcut and select regsiter anaconda as my default python 3,14 then Click on install

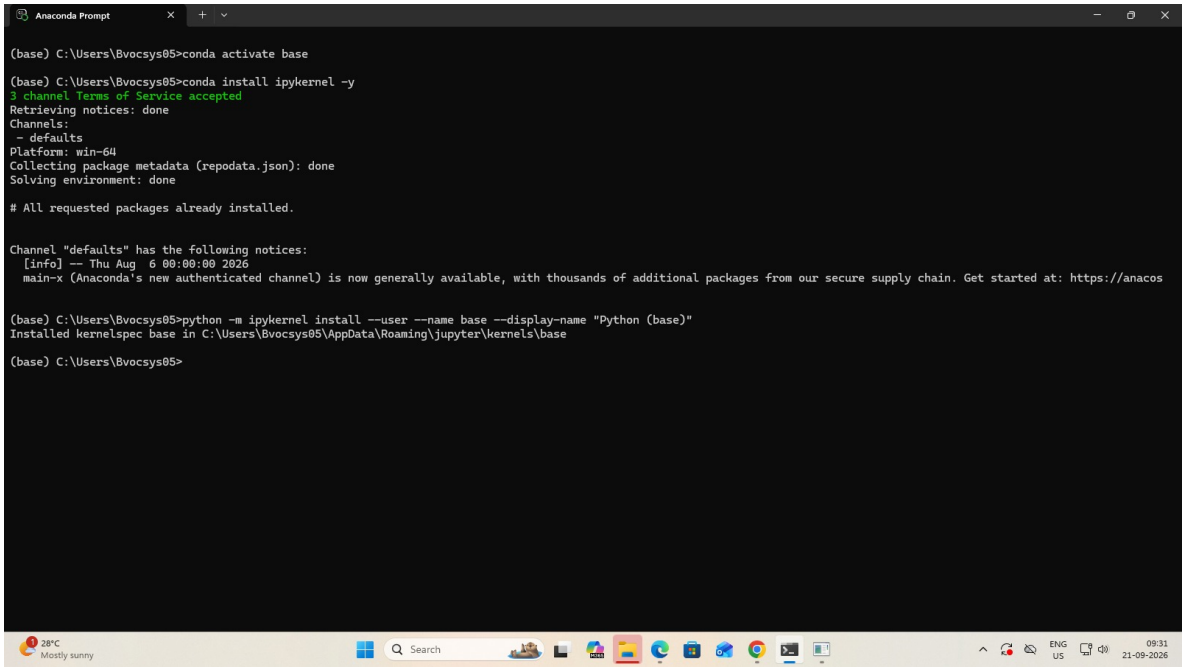


Step 8 After susesuful competion of installtion you should get this window

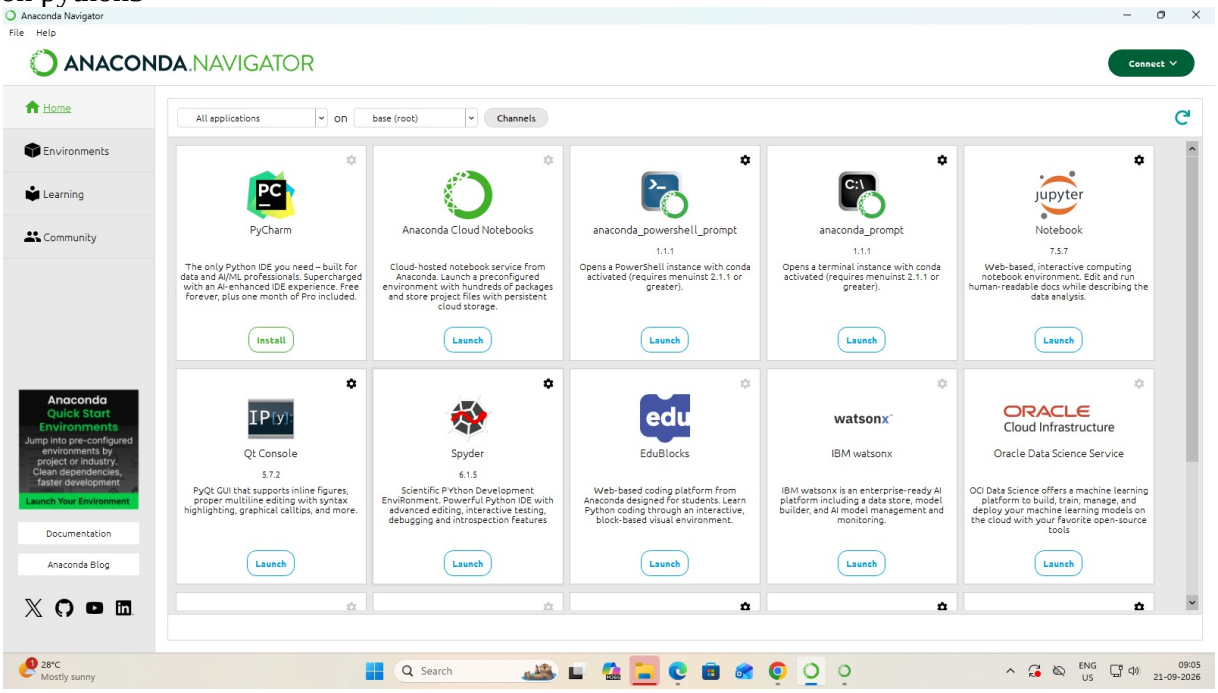


Step 9 Then open cmd and run this comands

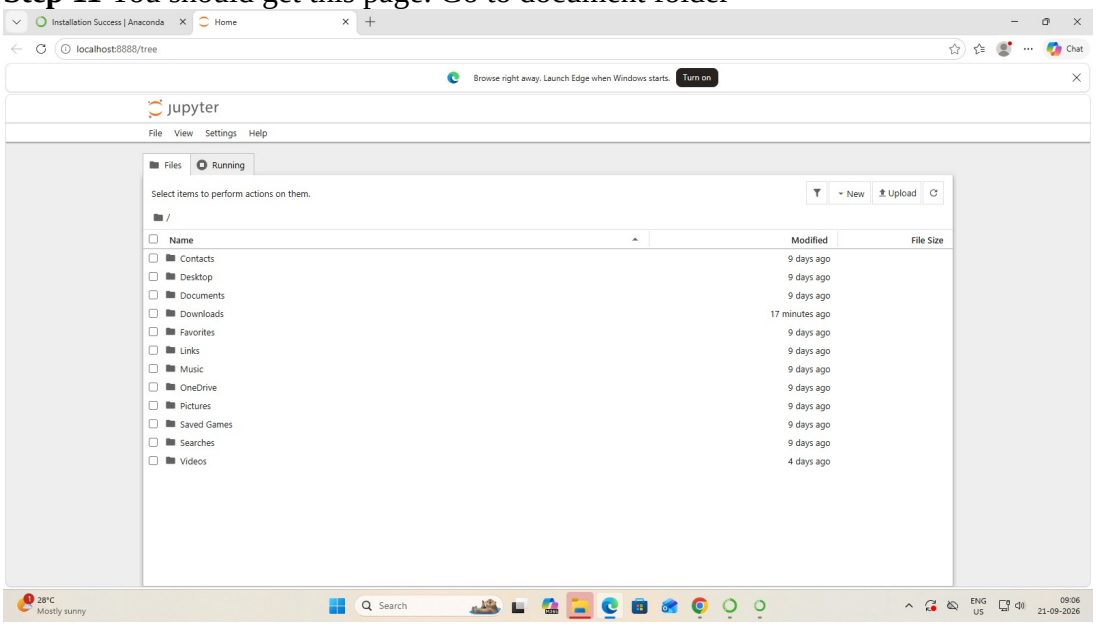
1) conda activate base
2) conda install ipykernel -y
3) python -m ipykernel install –user –name base –display-name “python (base)”



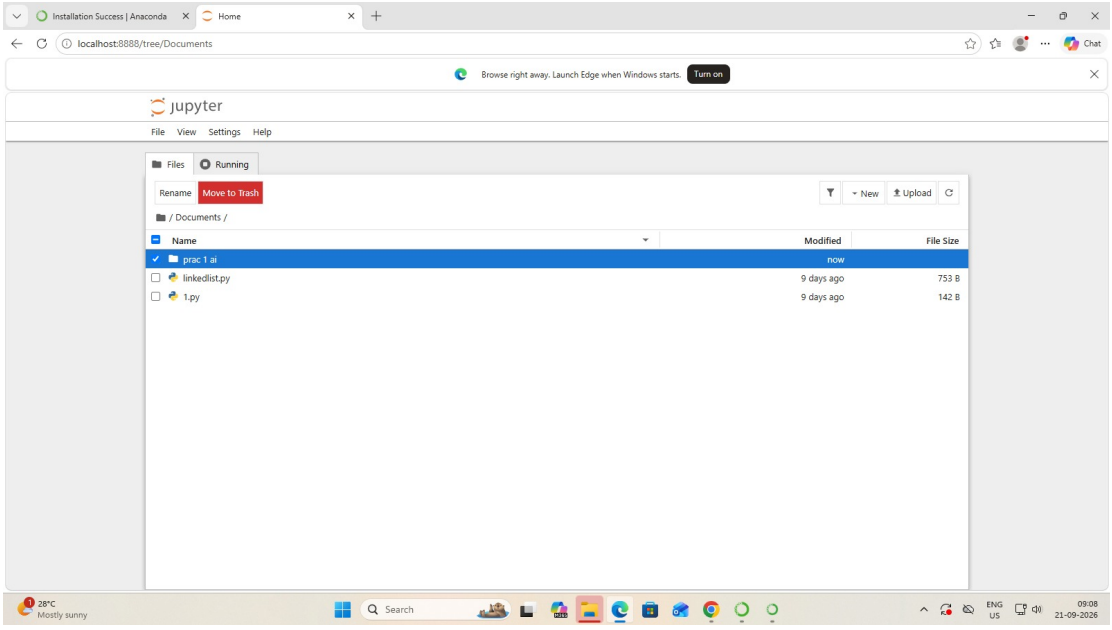
Step 10 Open Anaconda after opening you should get this page. Scroll down and click on launch button on python3



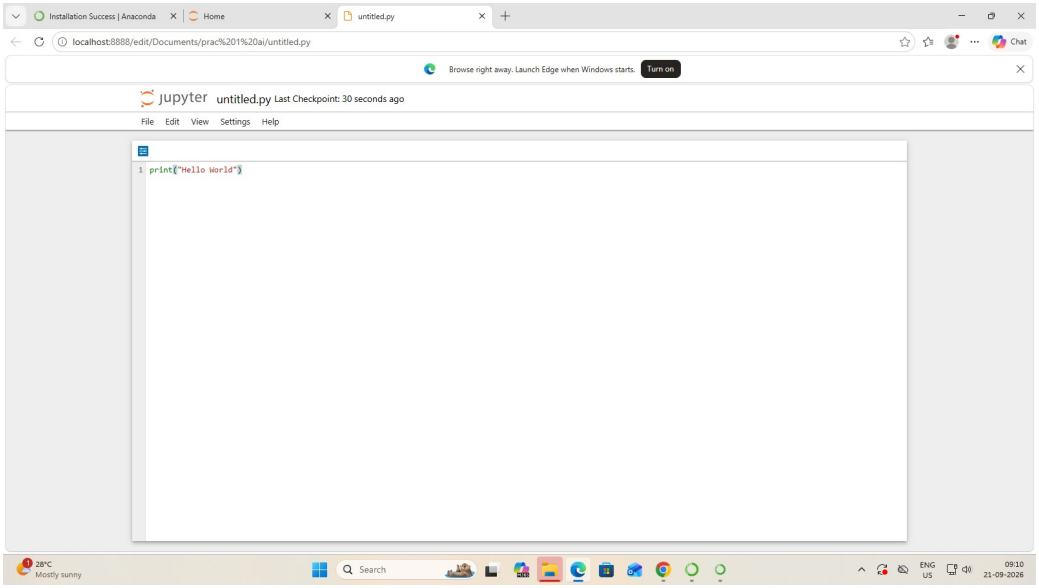
Step 11 You should get this page. Go to document folder



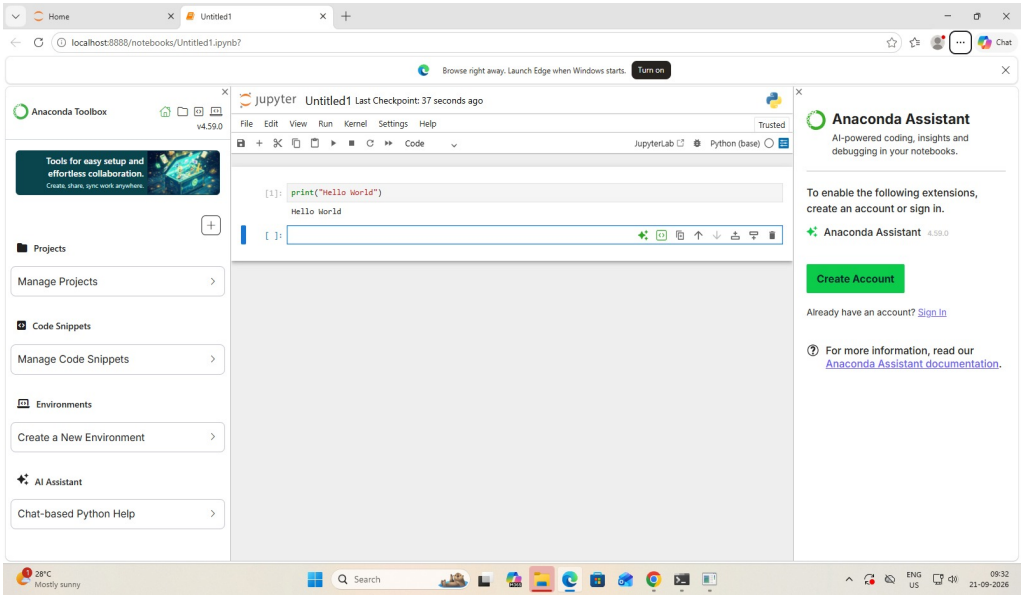
Step 12 Create new folder named “prac 1 ai” and open that folder



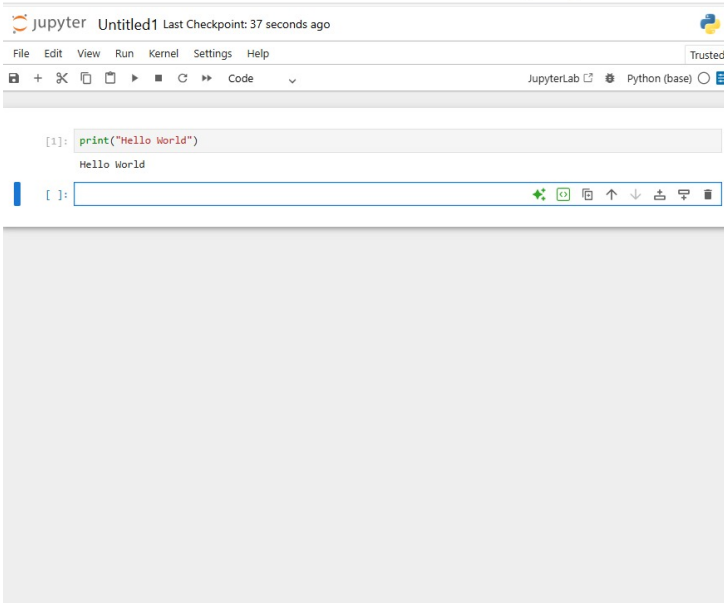
Step 13 Click new and select python3. Type print(“Hello World”)



Step 14 You should get “Hello world” output as shown below



Output:


jupyter Untitled1 Last Checkpoint: 37 seconds ago
File Edit View Run Kernel Settings Help
Trusted
Code
JupyterLab
Python (base)
[1]: print("Hello World")
Hello World
[ ]:

Practical-2

Aim: Build a simple Decision Tree malware detector using scikit-learn

Step 1 open Python idle and paste this code

```
import pandas as pd
from sklearn.model_selection import train_test_split
from sklearn.tree import DecisionTreeClassifier
from sklearn.metrics import accuracy_score, classification_report
# Create dataset
data = {
    'file_size': [120, 150, 130, 500, 600, 550,
                  110, 140, 520, 580, 125, 610],

    'entropy': [3.1, 3.4, 3.2, 7.2, 7.8, 7.5,
                3.0, 3.5, 7.4, 7.9, 3.3, 8.0],

    'suspicious_imports': [1, 2, 1, 8, 9, 7,
                           1, 2, 8, 9, 1, 10],

    'sections': [4, 5, 4, 8, 9, 8,
                 4, 5, 9, 8, 4, 10],

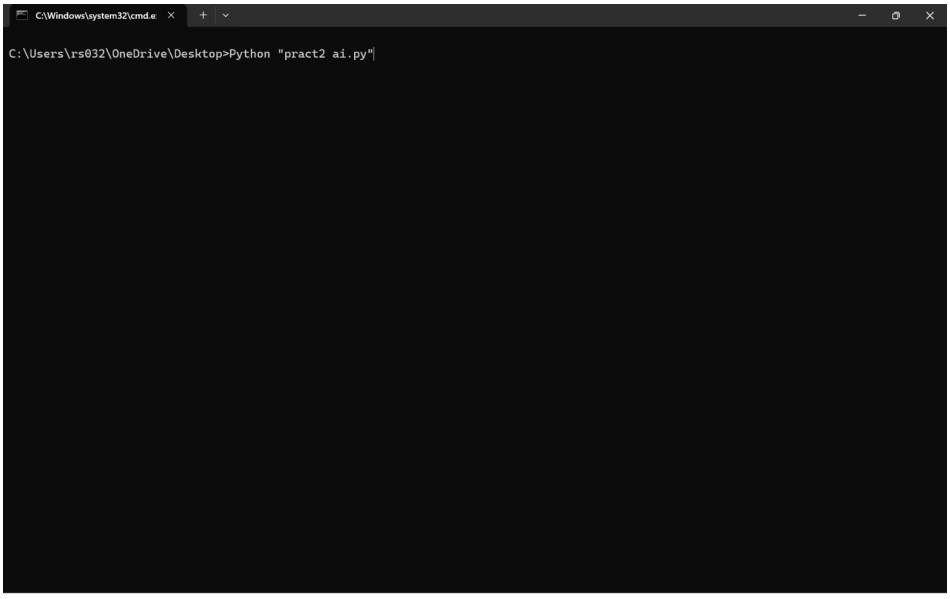
    'label': [0, 0, 0, 1, 1, 1,
              0, 0, 1, 1, 0, 1]
}

# Convert dataset into a DataFrame
df = pd.DataFrame(data)
print("Dataset:")
print(df)
# Separate features and labels
X = df.drop('label', axis=1)
y = df['label']
# Split data into training and testing
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.25, random_state=42
)
# Create Decision Tree
model = DecisionTreeClassifier(random_state=42)
# Train the model
model.fit(X_train, y_train)
# Make predictions
y_pred = model.predict(X_test)
# Calculate accuracy
accuracy = accuracy_score(y_test, y_pred)
print("\nAccuracy:", accuracy * 100, "%")
# Classification report
print("\nClassification Report:")
print(classification_report(y_test, y_pred))
# Test a new file
new_file = pd.DataFrame(
    [[550, 7.6, 8, 9]],
    columns=['file_size', 'entropy', 'suspicious_imports', 'sections']
)
prediction = model.predict(new_file)
if prediction[0] == 1:
    print("New File Prediction: Malware")
else:
    print("New File Prediction: Benign")
```

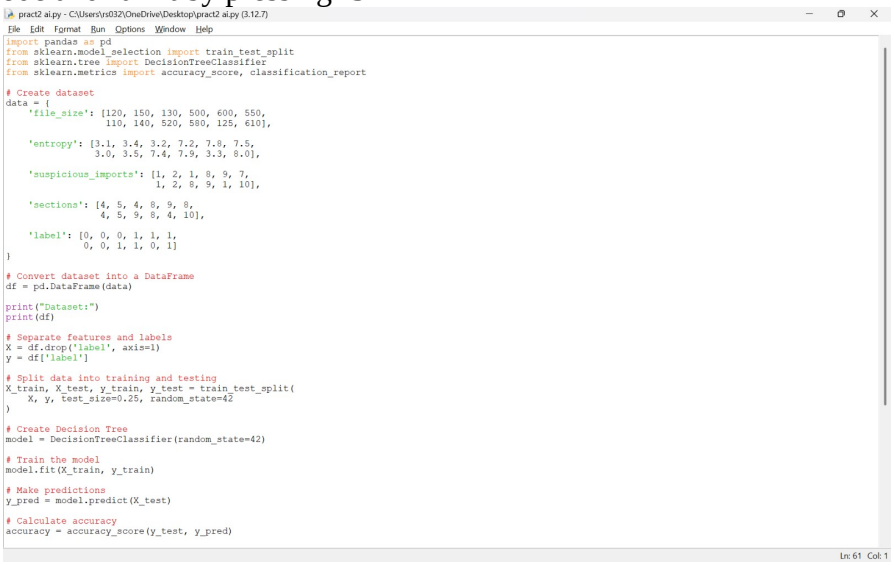
code explanation

- The program uses Python libraries such as Pandas and Scikit-learn to build a simple Decision Tree malware detector.
- It creates a dataset containing file size, entropy, suspicious imports, and number of sections.
- The label `0` represents a benign file, while `1` represents malware.
- The dataset is divided into 75% training data and 25% testing data.
- The Decision Tree model learns patterns from the training data and predicts the classes of test files.
- The model's performance is evaluated using accuracy, precision, recall, and F1-score.
- Finally, a new file is tested, and the program displays whether it is predicted as malware or benign.
- This practical demonstrates the basic application of machine learning in cybersecurity for malware detection.

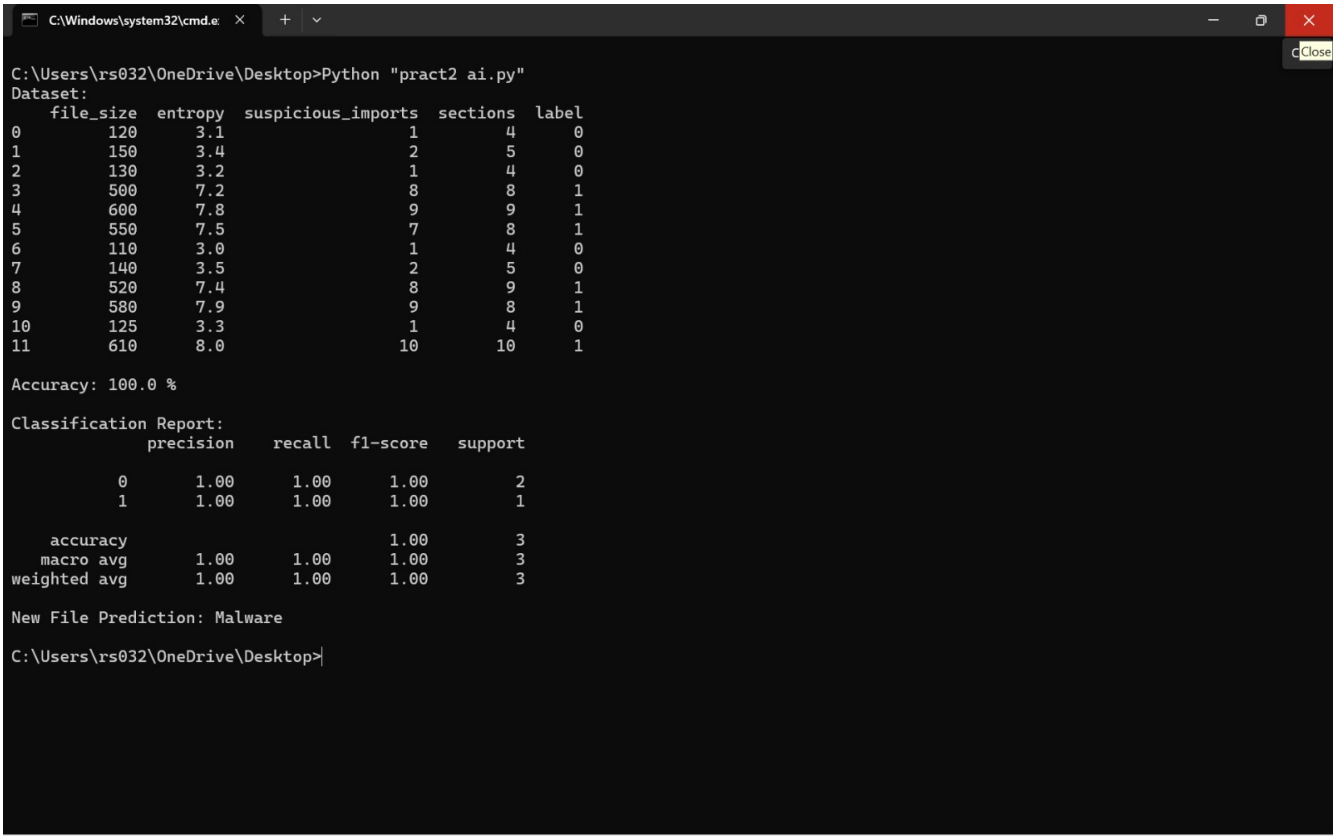
Step 2 Install required libraries by running following comands in cmd
pip install pandas
pip install sklearn



Step 3 save the code and run it by pressing f5



Step 4 If the code execution completed successfully you should get this as the final output



Conclusion:

The Decision Tree malware detector was successfully developed using Python and Scikit-learn. The model uses file features such as size, entropy, suspicious imports, and number of sections to classify files as malware or benign. Its performance is evaluated using accuracy and a classification report. This practical demonstrates how machine learning can be applied in cybersecurity to identify potentially malicious files and improve basic malware detection.

PRACTICAL 3 REPORT

Network Intrusion Detection using Decision Tree and Isolation Forest

1. Aim

To detect and classify suspicious network activity using machine learning, and to compare a supervised classification approach (Decision Tree) with an unsupervised anomaly-detection approach (Isolation Forest).

2. Software and Libraries Required

- Google Colab / Jupyter Notebook
- Python 3
- NumPy – numerical data generation and calculations
- Pandas – data handling and DataFrame operations
- Matplotlib – graphs and visualization
- Scikit-learn – Decision Tree, Isolation Forest, train/test split, and evaluation metrics

3. Dataset Description

For demonstration, a synthetic network-traffic dataset is generated using NumPy. It contains 525 records: 500 normal records and 25 attack records.

Feature	Meaning	Normal pattern	Attack pattern
duration	Length of the network connection	Lower values	Higher values
bytes_sent	Amount of data sent	Around 500	Around 5000
connections	Number of connections	Around 10	Around 120
label	Class of traffic	0 = Normal	1 = Attack

4. Step-by-Step Procedure

1. Import the required Python libraries.
2. Generate 500 normal network records and 25 attack records using random numerical distributions.
3. Combine the two datasets into one Pandas DataFrame.
4. Display the first few records and count the labels to verify the dataset.
5. Plot bytes sent against the number of connections to visually observe the difference between normal and attack traffic.
6. Select duration, bytes_sent, and connections as input features (X), and label as the target (y).
7. Split the dataset into 70% training data and 30% testing data using train_test_split.
8. Train a Decision Tree classifier with max_depth=3.
9. Predict the class of the test records and evaluate the model using a confusion matrix and classification report.
10. Train an Isolation Forest using only the feature data, without using labels during model fitting.
11. Convert Isolation Forest output into 0/1 format and compare the detected anomalies with the known labels.
12. Plot the Isolation Forest predictions to visually show normal records and detected anomalies.

5. Implementation Steps with Code

Step 1 – Import libraries

The required libraries are imported first. Scikit-learn provides the machine-learning algorithms and evaluation functions.

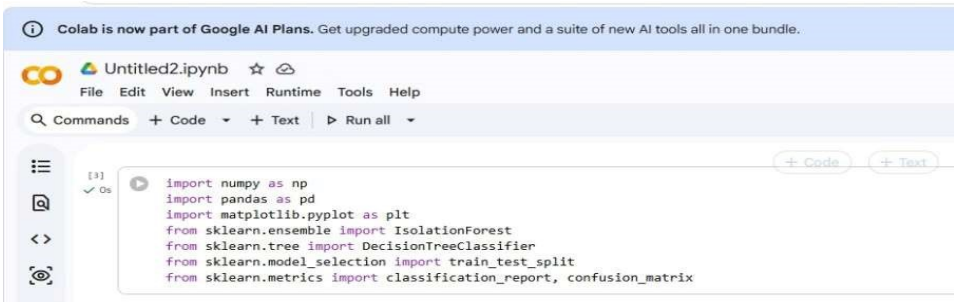


Figure 1: Importing Python and machine-learning libraries.

Step 2 – Generate the dataset

Normal traffic is generated with smaller values, while attack traffic is generated with much larger values for bytes sent and connections. The records are then combined.

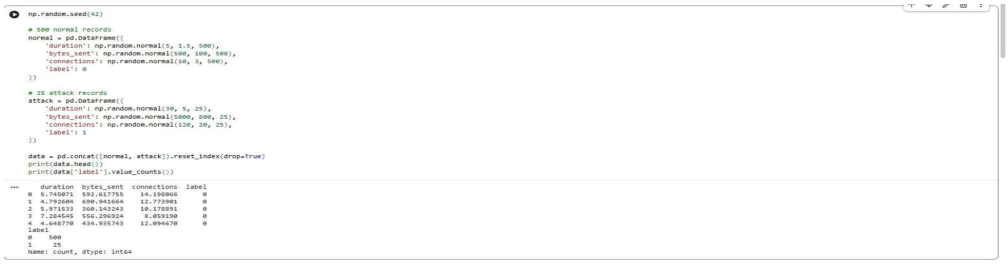


Figure 2: Generating normal and attack network records.

Step 3 – Visualize the data

A scatter plot is used to compare bytes sent and connections. Blue points represent normal traffic and red points represent attack traffic.

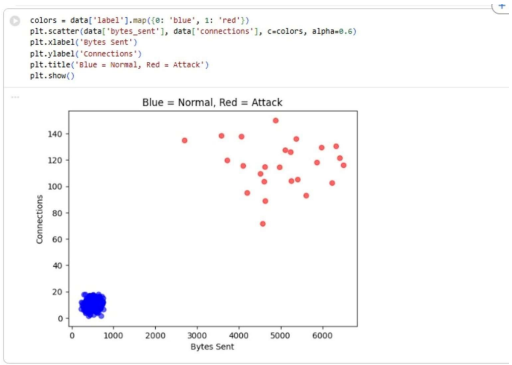


Figure 3: Scatter plot showing normal and attack traffic.

Step 4 – Split training and testing data

The features are separated from the target label. The data is divided into 70% training and 30% testing data. stratify=y keeps the class distribution balanced between the two sets.

```
X = data[['duration', 'bytes_sent', 'connections']]
y = data['label']

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.3, random_state=42, stratify=y
)
```

Figure 4: Preparing features and splitting the dataset.

Step 5 – Train and evaluate the Decision Tree

A Decision Tree with a maximum depth of 3 is trained. The test predictions are evaluated using a confusion matrix and classification report.

```
clf = DecisionTreeClassifier(max_depth=3, random_state=42)
clf.fit(X_train, y_train)
pred = clf.predict(X_test)

print(confusion_matrix(y_test, pred))
print(classification_report(y_test, pred))
```

```
[[150   0]
 [  0   8]]
              precision    recall  f1-score   support

           0       1.00      1.00      1.00       150
           1       1.00      1.00      1.00         8

    accuracy                           1.00       158
   macro avg       1.00      1.00      1.00       158
weighted avg       1.00      1.00      1.00       158
```

Figure 5: Decision Tree classification and evaluation.

Step 6 – Isolation Forest anomaly detection

Isolation Forest is trained on the feature data without using the labels. The model marks normal observations as 1 and anomalies as -1 internally; these values are converted into the 0/1 format used in the experiment.

```
iso = IsolationForest(n_estimators=100, contamination=0.05, random_state=42)
iso.fit(X)  # Unsupervised: no labels used

data['pred'] = iso.predict(X)
data['pred'] = data['pred'].map({1: 0, -1: 1})  # Convert to 0/1 format

print(confusion_matrix(data['label'], data['pred']))
print(classification_report(data['label'], data['pred']))
```

```
[[498   2]
 [  0  25]]
              precision    recall  f1-score   support

           0       1.00      1.00      1.00       500
           1       0.93      1.00      0.96        25

    accuracy                           1.00       525
   macro avg       0.96      1.00      0.98       525
weighted avg       1.00      1.00      1.00       525
```

Figure 6: Isolation Forest implementation and evaluation.

Step 7 – Visualize Isolation Forest results

The predicted class is plotted against bytes sent and connections. The attack-like records are clearly separated from the dense cluster of normal traffic.

```
colors = data['pred'].map({0: 'blue', 1: 'red'})
plt.scatter(data['bytes_sent'], data['connections'], c=colors, alpha=0.6)
plt.xlabel('Bytes Sent')
plt.ylabel('Connections')
plt.title('Isolation Forest: Red = Detected Anomaly')
plt.show()
```

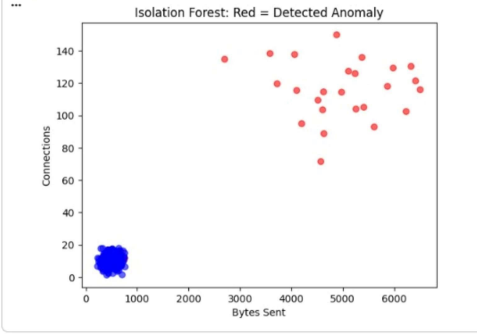


Figure 7: Isolation Forest anomaly-detection visualization.

6. Results

Decision Tree results:

Metric	Class 0 – Normal	Class 1 – Attack
Precision	1.00	1.00
Recall	1.00	1.00
F1-score	1.00	1.00
Support	150	8

Overall Decision Tree accuracy: 100% on the 158-record test set.

Isolation Forest results:

Metric	Class 0 – Normal	Class 1 – Attack
Precision	1.00	0.93
Recall	1.00	1.00
F1-score	1.00	0.96
Support	500	25

The Isolation Forest confusion matrix was [[498, 2], [0, 25]], meaning 498 normal records were correctly identified, 2 normal records were flagged as anomalies, and all 25 attack records were detected.

7. Conclusion

The practical successfully demonstrates machine-learning based network intrusion detection. A Decision Tree was used as a supervised classifier and achieved 100% accuracy on the synthetic test set. Isolation Forest was then used as an unsupervised anomaly detector and successfully identified all 25 attack records, with 2 normal records incorrectly classified as anomalies. The experiment shows how both labelled and unlabelled machine-learning techniques can be applied to cybersecurity data.

PRACTICAL 4

Network Intrusion Detection using Decision Tree and Isolation Forest

1. Aim

To detect and classify suspicious network activity using machine learning, and to compare a supervised classification approach (Decision Tree) with an unsupervised anomaly-detection approach (Isolation Forest).

2. Software and Libraries Required

- Google Colab / Jupyter Notebook
- Python 3
- NumPy – numerical data generation and calculations
- Pandas – data handling and DataFrame operations
- Matplotlib – graphs and visualization
- Scikit-learn – Decision Tree, Isolation Forest, train/test split, and evaluation metrics

6. Dataset Description

For demonstration, a synthetic network-traffic dataset is generated using NumPy. It contains 525 records: 500 normal records and 25 attack records.

Feature	Meaning	Normal pattern	Attack pattern
duration	Length of the network connection	Lower values	Higher values
bytes_sent	Amount of data sent	Around 500	Around 5000
connections	Number of connections	Around 10	Around 120
label	Class of traffic	0 = Normal	1 = Attack

8. Implementation Steps with Code

Step 1 – Import libraries

The required libraries are imported first. Scikit-learn provides the machine-learning algorithms and evaluation functions.



Figure 1: Importing Python and machine-learning libraries.

Step 2 – Generate the dataset

Normal traffic is generated with smaller values, while attack traffic is generated with much larger values for bytes sent and connections. The records are then combined.

```
np.random.seed(42)

# 500 normal records
normal = pd.DataFrame({
    'duration': np.random.normal(5, 1.5, 500),
    'bytes_sent': np.random.normal(500, 100, 500),
    'connections': np.random.normal(10, 3, 500),
    'label': 0
})

# 25 attack records
attack = pd.DataFrame({
    'duration': np.random.normal(30, 5, 25),
    'bytes_sent': np.random.normal(5000, 800, 25),
    'connections': np.random.normal(120, 20, 25),
    'label': 1
})

data = pd.concat([normal, attack]).reset_index(drop=True)
print(data.head())
print(data['label'].value_counts())
```

```
   duration  bytes_sent  connections  label
0  5.745071  592.617755    14.198066      0
1  4.792604  690.941664    12.773901      0
2  5.971533  360.143243    10.178891      0
3  7.284545  556.296924     8.059190      0
4  4.648770  434.935743    12.094670      0
label
0    500
1     25
Name: count, dtype: int64
```

Figure 2: Generating normal and attack network records.

Step 3 – Visualize the data

A scatter plot is used to compare bytes sent and connections. Blue points represent normal traffic and red points represent attack traffic.

```
colors = data['label'].map({0: 'blue', 1: 'red'})
plt.scatter(data['bytes_sent'], data['connections'], c=colors, alpha=0.6)
plt.xlabel('Bytes Sent')
plt.ylabel('Connections')
plt.title('Blue = Normal, Red = Attack')
plt.show()
```

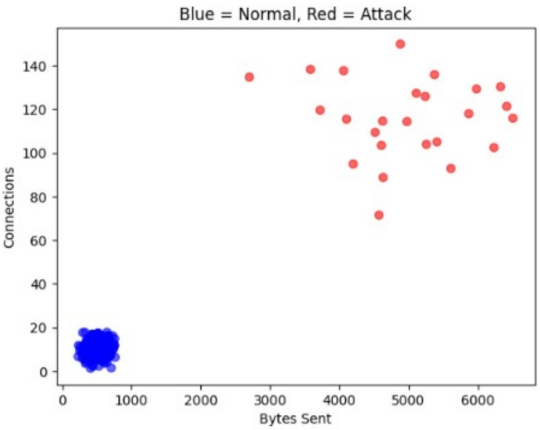


Figure 3: Scatter plot showing normal and attack traffic.

Step 4 – Split training and testing data

The features are separated from the target label. The data is divided into 70% training and 30% testing data. stratify=y keeps the class distribution balanced between the two sets.

```
X = data[['duration', 'bytes_sent', 'connections']]
y = data['label']

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.3, random_state=42, stratify=y
)
```

Figure 4: Preparing features and splitting the dataset.

Step 5 – Train and evaluate the Decision Tree

A Decision Tree with a maximum depth of 3 is trained. The test predictions are evaluated using a confusion matrix and classification report.

```
clf = DecisionTreeClassifier(max_depth=3, random_state=42)
clf.fit(X_train, y_train)
pred = clf.predict(X_test)

print(confusion_matrix(y_test, pred))
print(classification_report(y_test, pred))
```

```
[[150   0]
 [  0   8]]
              precision    recall  f1-score   support

           0       1.00      1.00      1.00       150
           1       1.00      1.00      1.00         8

    accuracy                           1.00       158
   macro avg       1.00      1.00      1.00       158
weighted avg       1.00      1.00      1.00       158
```

Figure 5: Decision Tree classification and evaluation.

Step 6 – Isolation Forest anomaly detection

Isolation Forest is trained on the feature data without using the labels. The model marks normal observations as 1 and anomalies as -1 internally; these values are converted into the 0/1 format used in the experiment.

```
iso = IsolationForest(n_estimators=100, contamination=0.05, random_state=42)
iso.fit(X)  # Unsupervised: no labels used

data['pred'] = iso.predict(X)
data['pred'] = data['pred'].map({1: 0, -1: 1})  # Convert to 0/1 format

print(confusion_matrix(data['label'], data['pred']))
print(classification_report(data['label'], data['pred']))
```

```
[[498   2]
 [  0  25]]
              precision    recall  f1-score   support

           0       1.00      1.00      1.00       500
           1       0.93      1.00      0.96        25

    accuracy                           1.00       525
   macro avg       0.96      1.00      0.98       525
weighted avg       1.00      1.00      1.00       525
```

Step 7 – Visualize Isolation Forest results

The predicted class is plotted against bytes sent and connections. The attack-like records are clearly separated from the dense cluster of normal traffic.

```
colors = data['pred'].map({0: 'blue', 1: 'red'})
plt.scatter(data['bytes_sent'], data['connections'], c=colors, alpha=0.6)
plt.xlabel('Bytes Sent')
plt.ylabel('Connections')
plt.title('Isolation Forest: Red = Detected Anomaly')
plt.show()
```

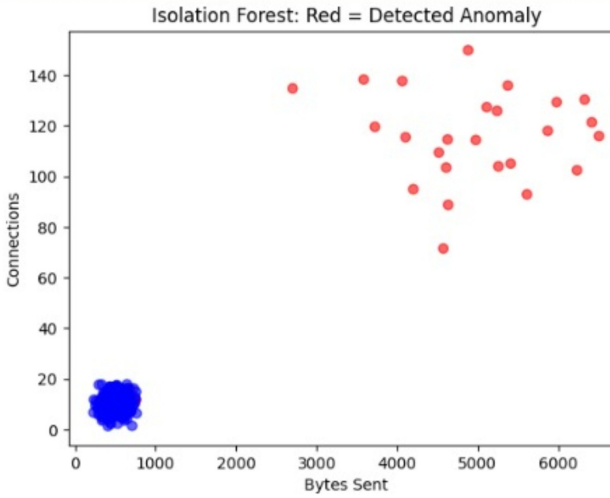


Figure 7: Isolation Forest anomaly-detection visualization.

9. Results

Decision Tree results:

Metric	Class 0 – Normal	Class 1 – Attack
Precision	1.00	1.00
Recall	1.00	1.00
F1-score	1.00	1.00
Support	150	8

Overall Decision Tree accuracy: 100% on the 158-record test set.

Isolation Forest results:

Metric	Class 0 – Normal	Class 1 – Attack
Precision	1.00	0.93
Recall	1.00	1.00

F1-score	1.00	0.96
Support	500	25

The Isolation Forest confusion matrix was [[498, 2], [0, 25]], meaning 498 normal records were correctly identified, 2 normal records were flagged as anomalies, and all 25 attack records were detected.

10. Interpretation

- The scatter plot shows a strong separation between normal and attack traffic.
- Attack records have substantially higher bytes_sent and connections in this synthetic dataset.
- The Decision Tree achieved perfect classification on the available test data because the generated classes are highly separated.
- Isolation Forest detected all 25 attack records, while incorrectly flagging 2 normal records as anomalies.
- The practical demonstrates both supervised classification and unsupervised anomaly detection.

11. Advantages

- Simple and easy to implement.
- Decision Trees are easy to understand and interpret.
- Isolation Forest can detect unusual behaviour without requiring attack labels during training.
- Graphs make the difference between normal and suspicious traffic easy to visualize.

12. Limitations

- The dataset is synthetic and highly separated, so the 100% Decision Tree accuracy should not be treated as real-world performance.
- Real network traffic contains noise, overlapping classes, categorical fields, and many different attack types.
- A practical IDS should be tested on realistic datasets and evaluated using additional metrics such as ROC-AUC, precision-recall, false-positive rate, and detection rate.

14. Conclusion

The practical successfully demonstrates machine-learning based network intrusion detection. A Decision Tree was used as a supervised classifier and achieved 100% accuracy on the synthetic test set. Isolation Forest was then used as an unsupervised anomaly detector and successfully identified all 25 attack records, with 2 normal records incorrectly classified as anomalies. The experiment shows how both labelled and unlabelled machine-learning techniques can be applied to cybersecurity data.

PRACTICAL 5

Facial Recognition-Based Authentication

1. Practical Name

Facial Recognition-Based Authentication using PCA and K-Nearest Neighbors (KNN).

2. Aim

To implement a simple facial recognition-based authentication system using the Olivetti Faces dataset, Principal Component Analysis (PCA) for dimensionality reduction, and K-Nearest Neighbors (KNN) for user identification and authentication.

6. Software and Libraries Required

- Google Colab / Jupyter Notebook
- Python 3
- NumPy – numerical operations and threshold calculations
- Matplotlib – displaying face images and results
- Scikit-learn – Olivetti Faces dataset, PCA, KNN, and train-test splitting

7. Dataset Description

Item	Value	Meaning	Role
Total images	400	Total face images	Input dataset
People	40	40 different person IDs	Identity classes
Image size	64 × 64	Grayscale facial image	Visual input
Features	4096	64 × 64 pixel values	Original feature space
Authorized users	Persons 0–4	5 users	Enrolled identities
Impostors	Persons 5–39	35 users	Unauthorized identities

8. Implementation and Evidence

Step 1 – Import libraries

The first cell imports the libraries required for dataset loading, visualization, dimensionality reduction, classification, and dataset splitting.

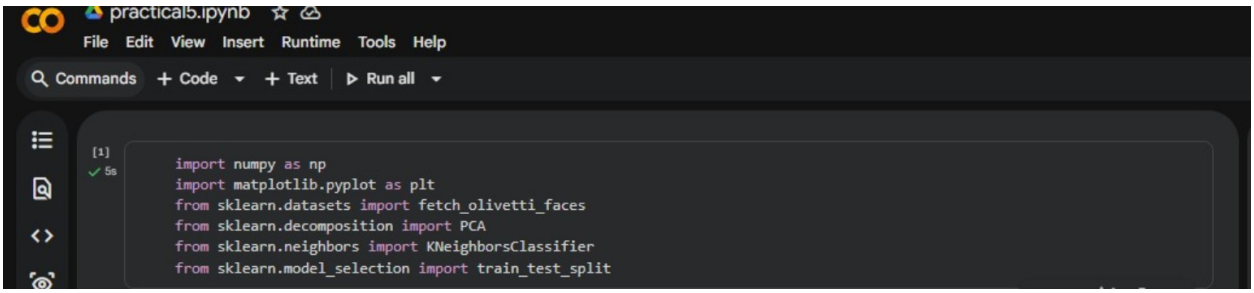


Figure 1: Importing the required Python libraries.

Step 2 – Load and visualize the dataset

The Olivetti Faces dataset is loaded. Each face is represented by 4096 pixel values. The output shows 400 images belonging to 40 people, and sample faces are displayed.

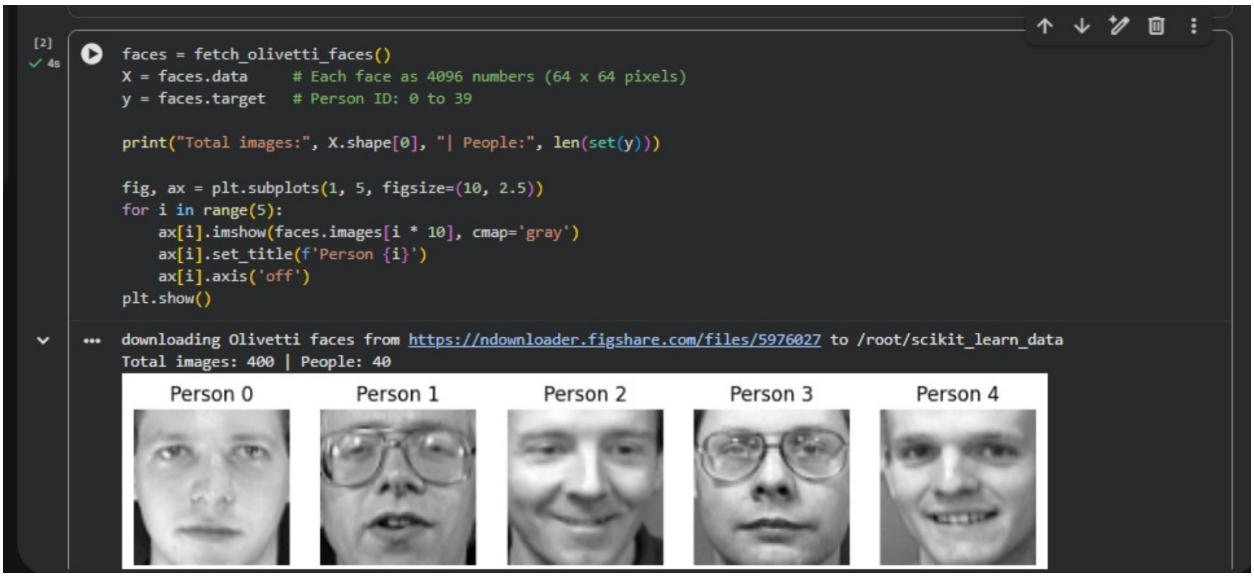
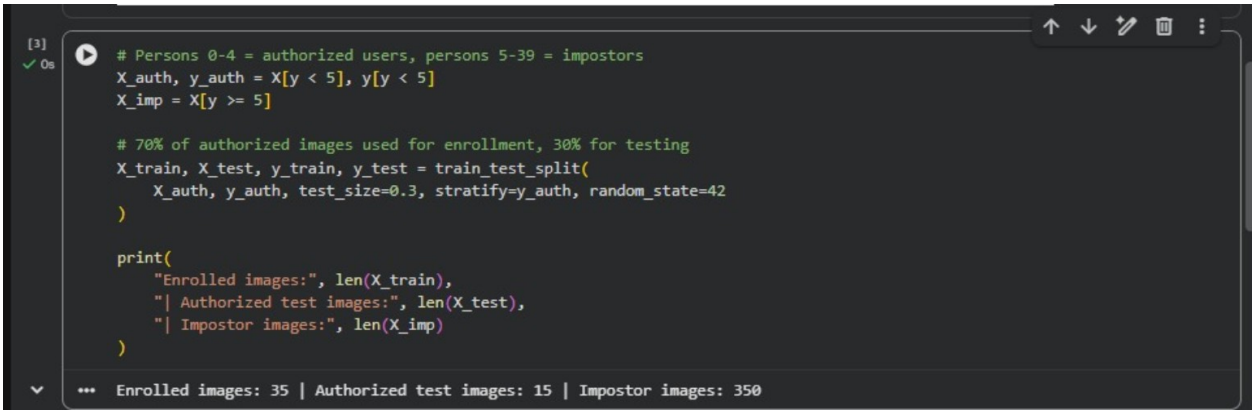


Figure 2: Loading the Olivetti Faces dataset and displaying sample faces.

Step 3 – Create enrollment and impostor sets

Persons 0–4 are considered authorized users. Their images are split into enrollment and authorized test sets. Persons 5–39 are considered impostors.



Observed result:

Enrolled images = 35 | Authorized test images = 15 | Impostor images = 350

Step 4 – PCA and KNN

PCA reduces the 4096-dimensional face representation to 20 components. KNN is then trained on the reduced enrollment data using one nearest neighbor.

```
pca = PCA(n_components=20, random_state=42)
Xtr = pca.fit_transform(X_train)  # Learn from enrolled faces only
Xte = pca.transform(X_test)
Ximp = pca.transform(X_imp)

knn = KNeighborsClassifier(n_neighbors=1)
knn.fit(Xtr, y_train)

print("Identification accuracy on authorized users:", knn.score(Xte, y_test))
```

```
Identification accuracy on authorized users: 1.0
```

Figure 4: PCA dimensionality reduction and KNN-based identification.

Observed identification accuracy on authorized users: 1.0 (100%).

Step 5 – Calculate authentication threshold

The distance from each enrolled face to its closest other enrolled face is calculated. The 95th percentile of these distances is selected as the authentication threshold.

```
# Distance from each enrolled face to its closest OTHER enrolled face
d_train, _ = knn.kneighbors(Xtr, n_neighbors=2)
threshold = np.percentile(d_train[:, 1], 95)

print("Threshold:", round(threshold, 2))
```

```
Threshold: 8.14
```

Figure 5: Calculating the authentication distance threshold.

Observed threshold: 8.14.

Step 6 – Analyze FRR and FAR

Different threshold values are tested. FRR (False Rejection Rate) represents genuine users that are incorrectly rejected. FAR (False Acceptance Rate) represents impostors that are incorrectly accepted.

```
d_auth = knn.kneighbors(Xte, n_neighbors=1)[0][:, 0]  # Authorized users' distances
d_imp = knn.kneighbors(Ximp, n_neighbors=1)[0][:, 0]  # Impostors' distances

print("Threshold | FRR (real user rejected) | FAR (impostor accepted)")
for m in [0.6, 0.8, 1.0, 1.2, 1.5]:
    t = threshold * m
    frr = np.mean(d_auth > t) * 100
    far = np.mean(d_imp <= t) * 100
    print(f"{t:8.2f} | {frr:12.1f}% | {far:17.1f}%")
```

```
Threshold | FRR (real user rejected) | FAR (impostor accepted)
    4.88 |         33.3% |              21.1%
    6.51 |          6.7% |              76.6%
    8.14 |          0.0% |              97.4%
    9.76 |          0.0% |              99.7%
   12.20 |          0.0% |             100.0%
```

9. FRR and FAR Results

Threshold	FRR – Real User Rejected	FAR – Impostor Accepted
4.88	33.3%	21.1%
6.51	6.7%	76.6%
8.14	0.0%	97.4%
9.76	0.0%	99.7%
12.20	0.0%	100.0%

The results show an important security trade-off. Increasing the threshold makes it easier for genuine users to be accepted, reducing FRR, but it also increases the chance of accepting impostors, thereby increasing FAR.

10. Step 7 – Authentication Function

The authentication function first transforms the input face using the previously fitted PCA model. It then finds the nearest enrolled face using KNN and obtains its distance. If the distance is less than or equal to the threshold, the system returns ACCEPTED and identifies the nearest user. Otherwise, it returns REJECTED.

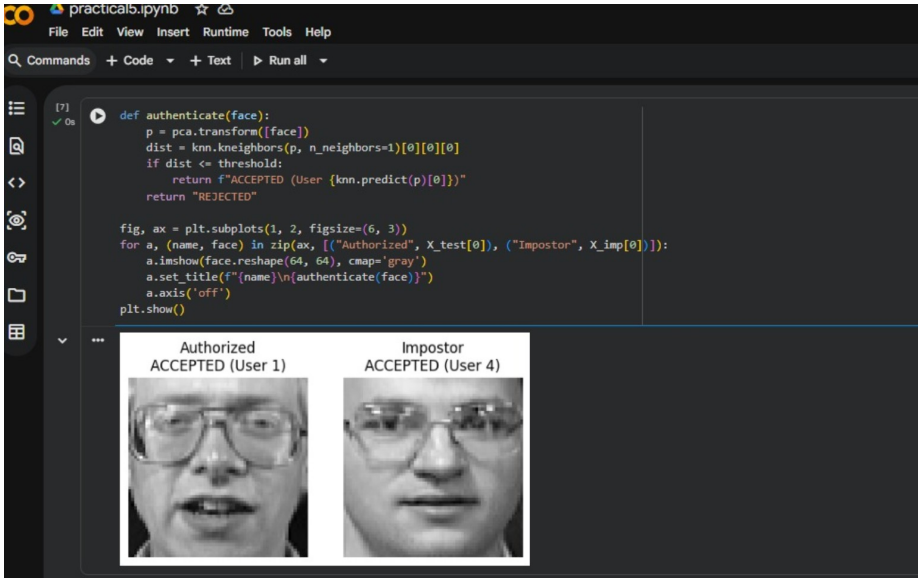


Figure 7: Authentication function tested with an authorized face and an impostor face.

In the displayed test, both the authorized sample and the impostor sample were accepted. This is consistent with the very high FAR measured at the selected threshold and is an important observation for evaluating the security of the threshold.

11. Results

- The Olivetti Faces dataset contained 400 images from 40 people.
- Five users (Persons 0–4) were designated as authorized users.
- 35 authorized images were used for enrollment and 15 for authorized testing.
- 350 images from Persons 5–39 were treated as impostors.
- PCA reduced the original 4096 pixel features to 20 components.
- KNN achieved 100% identification accuracy on the authorized test images.
- The calculated threshold was 8.14.
- At threshold 8.14, FRR was 0.0% but FAR was 97.4%.
- The demonstration showed that a face from an impostor group could be accepted, indicating that the threshold/setup is not secure enough for real authentication.

12. Interpretation

The practical demonstrates the difference between identification and authentication. KNN can identify which enrolled user is most similar to an input face, but a separate acceptance threshold is required to decide whether the input should actually be trusted.

The experiment also demonstrates that a high identification accuracy does not automatically mean a secure authentication system. Although authorized-user identification reached 100%, the FAR was extremely high at the chosen threshold. Therefore, threshold selection and evaluation against impostor data are critical in biometric authentication.

13. Advantages

- Uses a standard face dataset and common machine-learning algorithms.
- PCA reduces computational complexity by lowering the number of features.
- KNN is simple and easy to understand.
- The threshold provides an explicit authentication decision.
- FRR and FAR provide useful security-performance measures.

14. Limitations

- The Olivetti dataset is small and controlled compared with real-world facial recognition systems.
- The experiment uses grayscale, aligned 64 × 64 images rather than unconstrained camera images.
- KNN is sensitive to the quality and representation of the feature space.
- The selected threshold produces a very high FAR in this experiment.
- A real authentication system should use a stronger face representation, anti-spoofing/liveness detection, and carefully validated thresholds.
- Biometric systems should protect stored templates because facial biometric information cannot simply be replaced like a password.

15. Conclusion

The practical successfully implemented a basic facial recognition-based authentication system using the Olivetti Faces dataset, PCA, and KNN. PCA reduced the face representation from 4096 pixel features to 20 components, while KNN achieved 100% identification accuracy on the authorized test set. A distance threshold of 8.14 was obtained from the enrolled faces. However, the FRR/FAR analysis showed that the selected threshold resulted in a very high false acceptance rate of 97.4%. This demonstrates that accurate identification alone is not sufficient for secure authentication and that threshold selection, impostor testing, and anti-spoofing measures are essential in practical biometric security systems.

PRACTICAL 6

KneighborsClassifier-Based Authentication

Aim

To authenticate users (genuine user or imposter) using the KNeighborsClassifier algorithm in Python.

Software and Tools Required

- Google Colab (free, runs in the browser)
- Python libraries: NumPy, Pandas, Scikit-learn

Procedure (Exact Steps)

1. Open Google Colab (colab.research.google.com) and sign in with your Google account.
2. Click **File → New notebook**.
3. Rename the notebook to **Practical_6** by clicking the name at the top left.
4. Type the code of each step below into a separate cell.
5. Run each cell with the **Run** button or **Shift + Enter**.
6. Take a screenshot of each cell with its output for the journal.

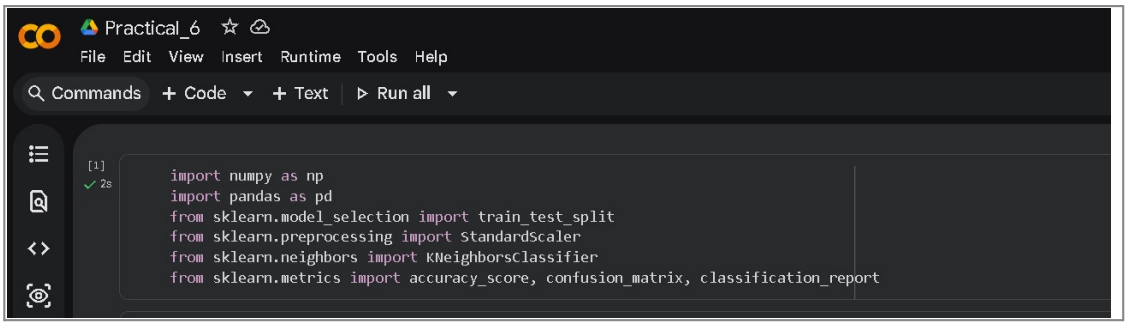
Step 1: Import the Libraries

```
import numpy as np import pandas as pd from
sklearn.model_selection import train_test_split from
sklearn.preprocessing import StandardScaler from
sklearn.neighbors import KNeighborsClassifier
from sklearn.metrics import accuracy_score, confusion_matrix, classification_report
```

Explanation:

- `import numpy as np`: loads NumPy, used for numbers and random data. `np` is a short name.
- `import pandas as pd`: loads Pandas, used to keep data in table form (DataFrame).
- `train_test_split`: splits data into a training part and a testing part.
- `StandardScaler`: brings all features to the same scale.
- `KNeighborsClassifier`: the KNN model we will use.
- `accuracy_score, confusion_matrix, classification_report`: tools to check how good the model is.

Output: No output is shown. A green tick means the cell ran successfully.



Screenshot 1: Output of Step 1

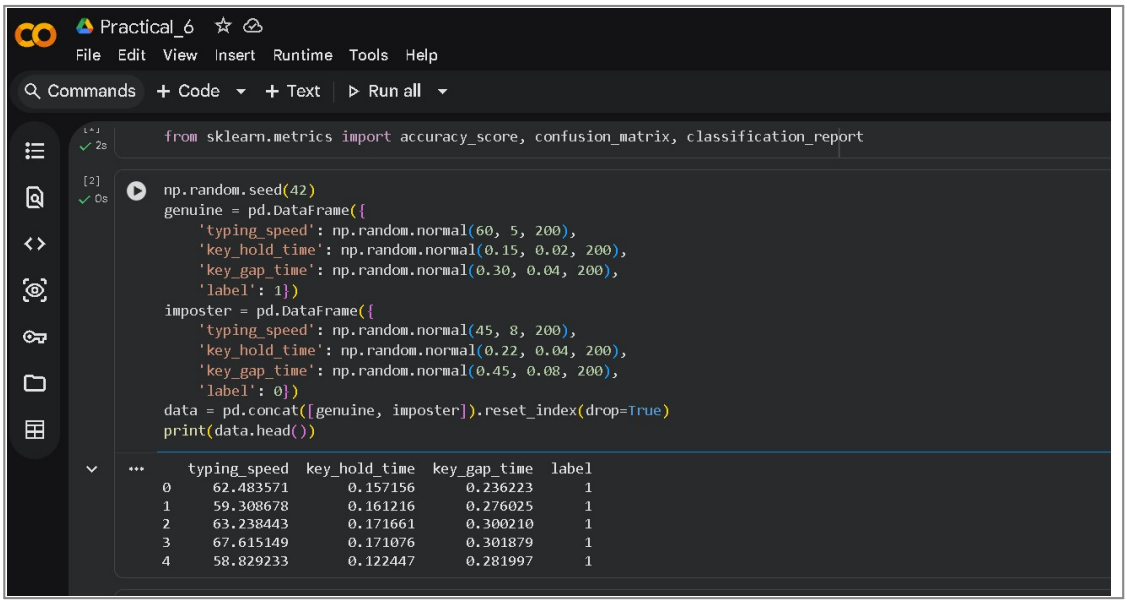
Step 2: Create the Dataset

```
np.random.seed(42) genuine =
pd.DataFrame({     'typing_speed':
np.random.normal(60, 5, 200),
    'key_hold_time': np.random.normal(0.15, 0.02, 200),
    'key_gap_time': np.random.normal(0.30, 0.04, 200),
'label': 1}) imposter =
pd.DataFrame({     'typing_speed':
np.random.normal(45, 8, 200),
    'key_hold_time': np.random.normal(0.22, 0.04, 200),
    'key_gap_time': np.random.normal(0.45, 0.08, 200),
'label': 0}) data = pd.concat([genuine,
imposter]).reset_index(drop=True) print(data.head())
```

Explanation:

- `np.random.seed(42)`: fixes the random numbers so you get the same data every time you run the code.
- `np.random.normal(60, 5, 200)`: creates 200 random numbers with an average of 60 and a spread of about 5.
- `genuine = pd.DataFrame({...})`: makes a table of 200 genuine-user records. The genuine user types fast (average 60), with a short hold time (0.15) and a short gap (0.30).
- `'label': 1`: marks every row in this table as genuine user.
- `imposter = ...`: makes another table of 200 imposter records. Imposters type slower (average 45), hold keys longer (0.22) and have bigger gaps (0.45). Their label is 0.
- `pd.concat([genuine, imposter])`: joins both tables into one table of 400 rows.
- `.reset_index(drop=True)`: renumbers the rows 0 to 399.
- `print(data.head())`: shows the first 5 rows to check the data.

Output: The first 5 rows are shown (all with label 1, because genuine rows come first).



Screenshot 2: Output of Step 2

Step 3: Separate Features and Label, then Split into Train and Test

```
X = data.drop('label', axis=1) y = data['label']
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42)
```

Explanation:

- `X = data.drop('label', axis=1)`: X holds the input features (typing speed, hold time, gap time). We remove the label column.
- `y = data['label']`: y holds only the answer (1 or 0) that the model should learn to predict.
- `train_test_split(...)`: splits the data randomly.

- `test_size=0.2`: 20% of the data (80 rows) is kept for testing and 80% (320 rows) is used for training.
- `random_state=42`: keeps the split the same every time.
- `X_train, y_train`: data used to teach the model.
- `X_test, y_test`: data kept hidden from the model, used later to check it.

Output: No output.



Screenshot 3: Output of Step 3

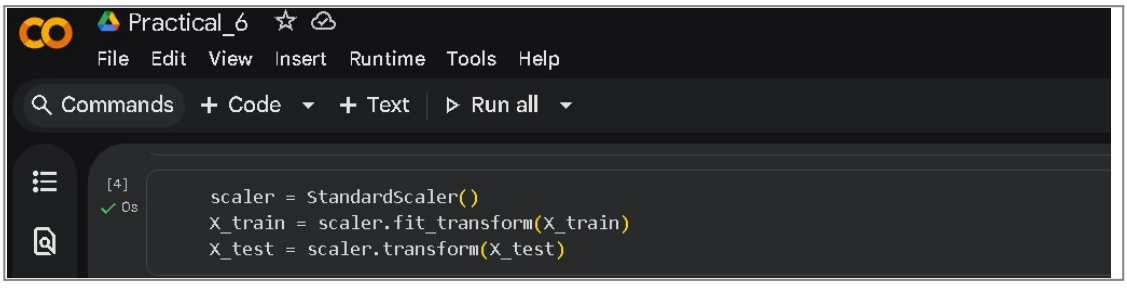
Step 4: Scale the Data

```
scaler = StandardScaler() X_train =
scaler.fit_transform(X_train)
X_test = scaler.transform(X_test)
```

Explanation:

- `scaler = StandardScaler()`: creates the scaler tool.
- `fit_transform(X_train)`: first *learns* the average and spread of the training data (fit), then *converts* the data to a common scale (transform).
- `transform(X_test)`: converts the test data using the **same** values learned from the training data. We do not fit on test data, because test data should stay unseen.
- **Why needed?** KNN uses distance. Without scaling, typing speed (big numbers) would control the result and the small features would be ignored.

Output: No output.



Screenshot 4: Output of Step 4

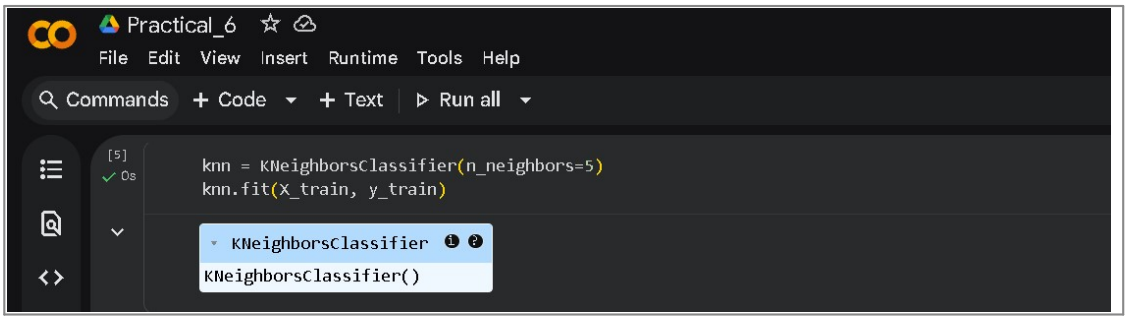
Step 5: Train the KNN Model

```
knn = KNeighborsClassifier(n_neighbors=5)
knn.fit(X_train, y_train)
```

Explanation:

- `KNeighborsClassifier(n_neighbors=5)`: creates the KNN model. It will look at the 5 nearest neighbors to decide.
- `knn.fit(X_train, y_train)`: trains the model. KNN simply stores the training data with its labels, and uses it later to find neighbors.

Output: The box showing KNeighborsClassifier() means the model is created and trained.



Screenshot 5: Output of Step 5

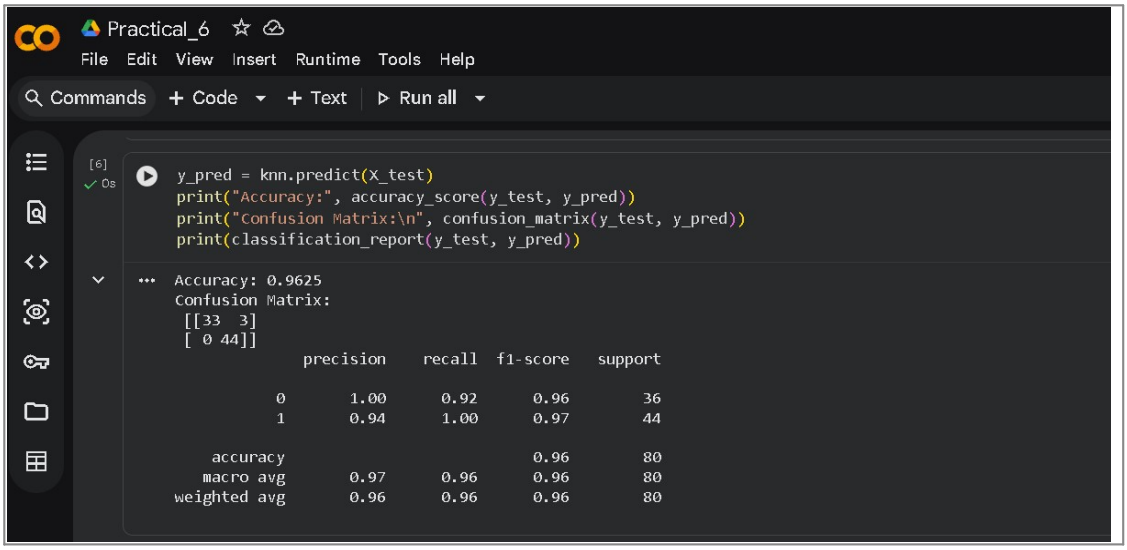
Step 6: Test the Model and Check Performance

```
y_pred = knn.predict(X_test) print("Accuracy:",
accuracy_score(y_test, y_pred)) print("Confusion Matrix:\n",
confusion_matrix(y_test, y_pred))
print(classification_report(y_test, y_pred))
```

Explanation:

- `knn.predict(X_test)`: the model predicts genuine (1) or imposter (0) for the 80 test rows. Predictions are stored in `y_pred`.
- `accuracy_score(y_test, y_pred)`: compares the real answers (`y_test`) with the predicted answers (`y_pred`) and gives the fraction predicted correctly.
- `confusion_matrix(...)`: a small table showing correct and wrong predictions.
- `classification_report(...)`: shows precision, recall and F1-score for each class.

Output obtained: Accuracy = 0.9625 and Confusion Matrix = [[33, 3], [0, 44]]



Screenshot 6: Output of Step 6

Classification report:

Class	Precision	Recall	F1-score	Support
0 (Imposter)	1.00	0.92	0.96	36
1 (Genuine)	0.94	1.00	0.97	44
Accuracy			0.96	80

Understanding the output:

Accuracy 0.9625 means the model was correct 96.25% of the time (77 out of 80). The confusion matrix is read as follows:

	Predicted Imposter	Predicted Genuine
Actual Imposter	33 (correctly rejected)	3 (wrongly accepted)
Actual Genuine	0 (wrongly rejected)	44 (correctly accepted)

- All 44 genuine users were accepted, and 33 of 36 imposters were rejected. Only 3 imposters were wrongly allowed in.
- **Precision:** out of all the predictions made for a class, how many were right.
- **Recall:** out of all the real members of a class, how many were found.
- **F1-score:** a balance of precision and recall.
- **Support:** the number of test samples in that class (36 imposters, 44 genuine).

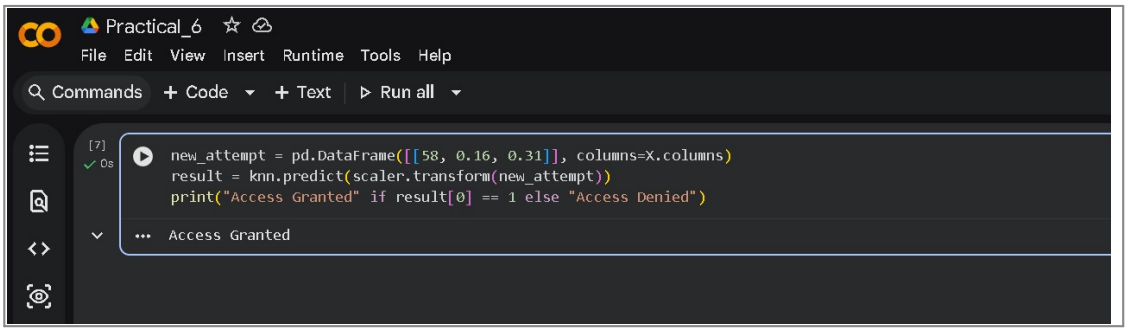
Step 7: Authenticate a New Login Attempt

```
new_attempt = pd.DataFrame([[58, 0.16, 0.31]], columns=X.columns)
result = knn.predict(scaler.transform(new_attempt)) print("Access
Granted" if result[0] == 1 else "Access Denied")
```

Explanation:

- `new_attempt = pd.DataFrame([[58, 0.16, 0.31]], columns=X.columns)`: creates one new login attempt with typing speed 58, hold time 0.16 and gap time 0.31. `columns=X.columns` gives it the same column names as the training data.
- `scaler.transform(new_attempt)`: scales the new attempt in the same way as the training data.
- `knn.predict(...)`: the model decides whether this attempt is genuine (1) or imposter (0).
- `result[0] == 1`: checks the answer. If it is 1, it prints "Access Granted", otherwise "Access Denied".

Output obtained: Access Granted. The values (58, 0.16, 0.31) are close to the genuine user's typing pattern, so the model accepted the login.



Screenshot 7: Output of Step 7

Result

The KNeighborsClassifier model was trained on 320 samples and tested on 80 samples. It reached an accuracy of **96.25%**. It accepted a new login attempt with genuine-user behaviour (Access Granted).

Conclusion

KNN can be used to authenticate users based on their typing behaviour. It separated genuine users from imposters with high accuracy. In the confusion matrix, no genuine user was rejected, but 3 imposters were wrongly accepted. In a real security system such errors must be kept very low, which can be improved with more data, more features, or a better value of K.

PRACTICAL 7

Fraud Detection Using a Sample Dataset and ML Models

Aim

To detect fraudulent transactions using machine learning models (Logistic Regression and Random Forest) on a sample dataset, and to compare the performance of the two models.

Software and Tools Required

- Google Colab (free, runs in the browser)
- Python libraries: Pandas, Scikit-learn

Dataset Used

2000 sample transactions are generated by the program. About 95% are normal (Class 0) and 5% are fraud (Class 1). Each transaction has 6 features. The values are scaled numeric scores created for practice, not real money values.

Column	Meaning
amount	Transaction amount score
time	Time of the transaction
location_risk	How unusual the location is
device_risk	How suspicious the device used is
num_transactions	Number of recent transactions
balance_change	Change in account balance
Class	0 = normal transaction, 1 = fraud

Procedure (Exact Steps)

1. Open Google Colab (colab.research.google.com) and sign in with your Google account.
2. Click **File → New notebook**.
3. Rename the notebook to **Practical_7** by clicking the name at the top left.
4. Type the code of each step below into a separate cell.
5. Run each cell with the **Run** button or **Shift + Enter**.
6. Take a screenshot of each cell with its output for the journal.

Step1: Import the Libraries

```
import pandas as pd from sklearn.datasets import
make_classification from sklearn.model_selection
import train_test_split from sklearn.preprocessing
import StandardScaler from sklearn.linear_model
import LogisticRegression from sklearn.ensemble
import RandomForestClassifier
from sklearn.metrics import accuracy_score, confusion_matrix, classification_report
```

Explanation:

- `import pandas as pd`: Pandas keeps data in table form (DataFrame).
- `make_classification`: creates a sample dataset for practice, so no download is needed.
- `train_test_split`: splits data into a training part and a testing part.
- `StandardScaler`: brings all features to the same scale.
- `LogisticRegression`: our first model.
- `RandomForestClassifier`: our second model.
- `accuracy_score, confusion_matrix, classification_report`: tools to check how good the models are.

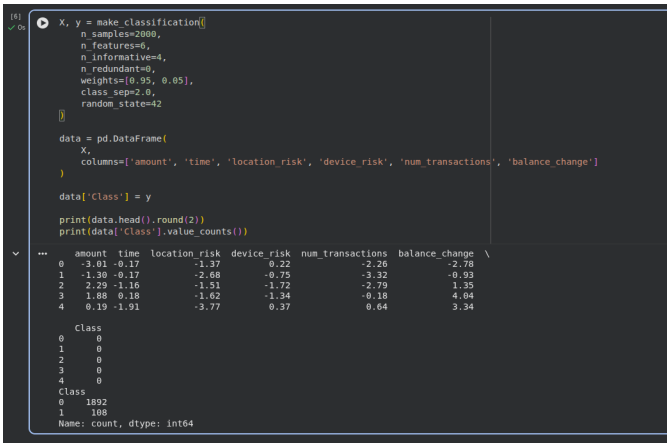
Output: No output is shown. A green tick means the cell ran successfully.

Step 2: Create the Sample Dataset

```
X, y = make_classification(n_samples=2000, n_features=6, n_informative=4,
n_redundant=0, weights=[0.95, 0.05],
class_sep=2.0, random_state=42) data = pd.DataFrame(X,
columns=[     'amount', 'time', 'location_risk',     'device_risk',
'num_transactions', 'balance_change']) data['Class'] = y
print(data.head().round(2)) print(data['Class'].value_counts())
```

Explanation:

- `make_classification(...)`: creates 2000 sample transactions. X holds the features and y holds the answer (0 or 1).
- `n_features=6`: each transaction has 6 features. `n_informative=4`: 4 of them really help in telling fraud from normal.
- `weights=[0.95, 0.05]`: 95% normal and 5% fraud, which makes the data imbalanced like real life.
- `class_sep=2.0`: how far apart the two classes are. A bigger value makes fraud easier to detect.
- `random_state=42`: gives the same data every time you run the code.
- `pd.DataFrame(X, columns=[...])`: puts the numbers in a table and names the 6 columns.
- `data['Class'] = y`: adds the answer column (0 = normal, 1 = fraud).
- `data.head()`: shows the first 5 rows. `.round(2)` limits the numbers to 2 decimals.
- `value_counts()`: counts how many normal and fraud transactions we have.



Understanding the output: Out of 2000 transactions, 1892 are normal and only 108 are fraud (about 5%). This shows the data is imbalanced.

Step 3: Split Features and Label, then Train and Test

```
X = data.drop('Class', axis=1) y = data['Class']
X_train, X_test, y_train, y_test = train_test_split(
X, y, test_size=0.2, random_state=42, stratify=y)
print("Training rows:", len(X_train)) print("Testing
rows :", len(X_test))
```

Explanation:

- `X = data.drop('Class', axis=1)`: X holds the input features. We remove the Class column.
- `y = data['Class']`: y holds only the answer the model must learn to predict.
- `test_size=0.2`: 20% (400 rows) is kept for testing and 80% (1600 rows) is used for training.
- `stratify=y`: keeps the same fraud percentage (about 5%) in both the training and testing parts. This is important because fraud is rare, and a plain random split might leave very few fraud cases in the test data.
- `random_state=42`: keeps the split the same every time.
- `len(X_train)`: counts the rows to confirm the split.

Output obtained:



Step 4: Scale the Data

```
scaler = StandardScaler() X_train =
scaler.fit_transform(X_train)
X_test = scaler.transform(X_test)
```

Explanation:

- `scaler = StandardScaler()`: creates the scaler tool.
- `fit_transform(X_train)`: first *learns* the average and spread of the training data, then *converts* the data to a common scale.
- `transform(X_test)`: converts the test data using the **same** values learned from the training data. We never learn from test data, because it must stay unseen.
- **Why needed?** Features with big numbers can dominate the ones with small numbers.
  Scaling makes them equal, and it helps Logistic Regression work properly.

Output: No output is shown. A green tick means the cell ran successfully.

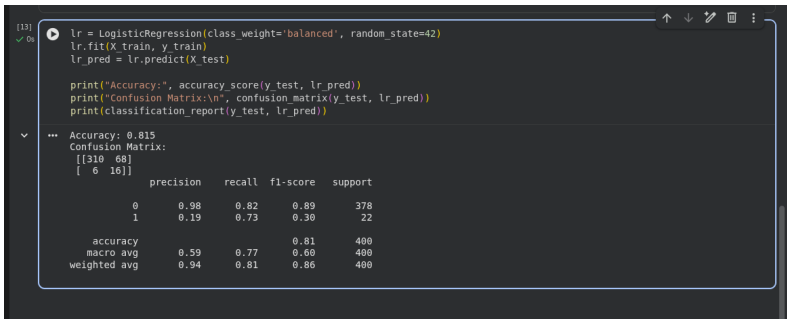
Step 5: Model 1: Logistic Regression

```
lr = LogisticRegression(class_weight='balanced', random_state=42)
lr.fit(X_train, y_train) lr_pred = lr.predict(X_test)
print("Accuracy:", accuracy_score(y_test, lr_pred))
print("Confusion Matrix:\n", confusion_matrix(y_test, lr_pred))
print(classification_report(y_test, lr_pred))
```

Explanation:

- `LogisticRegression(class_weight='balanced')`: creates the model. `balanced` gives extra importance to the rare fraud cases.
- `lr.fit(X_train, y_train)`: trains the model on the 1600 training rows.
- `lr.predict(X_test)`: predicts normal (0) or fraud (1) for the 400 test rows. The predictions are stored in `lr_pred`.
- `accuracy_score(...)`: the fraction of predictions that were correct.
- `confusion_matrix(...)`: a table of correct and wrong predictions.
- `classification_report(...)`: precision, recall and F1-score for each class.

Output obtained:



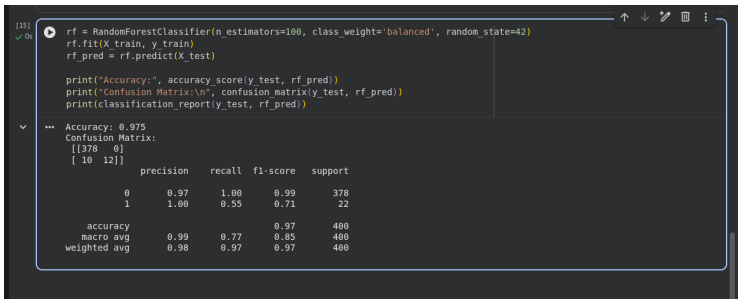
Step 6: Model 2: Random Forest

```
rf = RandomForestClassifier(n_estimators=100,
class_weight='balanced', random_state=42) rf.fit(X_train, y_train)
rf_pred = rf.predict(X_test) print("Accuracy:",
accuracy_score(y_test, rf_pred)) print("Confusion Matrix:\n",
confusion_matrix(y_test, rf_pred))
print(classification_report(y_test, rf_pred))
```

Explanation:

- `RandomForestClassifier(n_estimators=100, ...)`: creates a forest of 100 decision trees. `class_weight='balanced'` gives extra importance to fraud cases.
- `rf.fit(X_train, y_train)`: trains all 100 trees.
- `rf.predict(X_test)`: every tree votes and the majority vote is the final prediction, stored in `rf_pred`.
- The remaining three lines (accuracy, confusion matrix, report) are the same as in Step 5, so the two models can be compared fairly.

Output obtained:



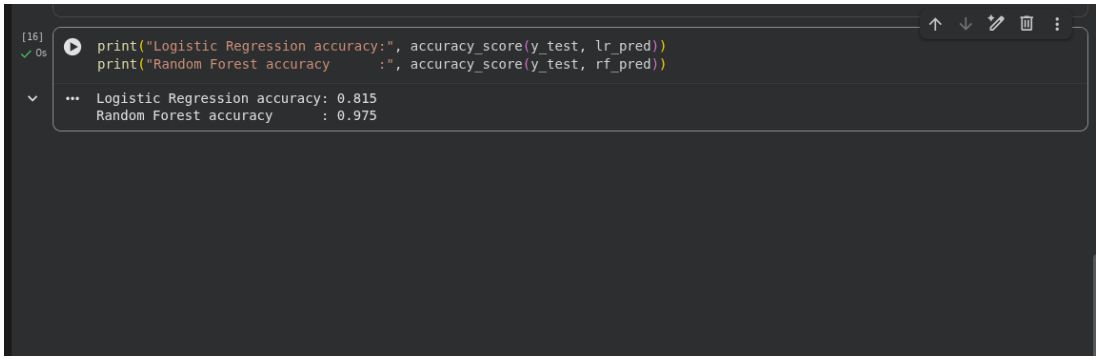
Step 7: Compare the Two Models

```
print("Logistic Regression accuracy:", accuracy_score(y_test, lr_pred))
print("Random Forest accuracy      :", accuracy_score(y_test, rf_pred))
```

Explanation:

- Prints the accuracy of both models one below the other, so they can be compared easily.

Output obtained:



Complete comparison (fraud class):

Measure	Logistic Regression	Random Forest
Accuracy	0.815	0.975
Precision (fraud)	0.19	1.00
Recall (fraud)	0.73	0.55
F1-score (fraud)	0.30	0.71
Frauds caught (of 22)	16	12
False alarms	68	0

Which model is better? Random Forest has the higher accuracy (97.5% against 81.5%) and the higher fraud F1-score (0.71 against 0.30), so overall it is the better model here. Logistic Regression catches more frauds but raises too many false alarms. In a real bank, the choice depends on what is more costly: missing a fraud or annoying a genuine customer.

Result

Both models were trained on 1600 transactions and tested on 400. Logistic Regression reached an accuracy of **81.5%** and Random Forest reached **97.5%**. Random Forest gave the better overall result, with a fraud F1-score of 0.71.

Conclusion

Machine learning models can detect fraudulent transactions in a sample dataset. Because fraud data is imbalanced, accuracy alone is not enough, and precision, recall and F1-score must also be checked. Random Forest performed better than Logistic Regression, but it still missed some frauds. Results can be improved with more data, more features, or techniques such as SMOTE and parameter tuning.

PRACTICAL 8

Implementing a Simple GAN for Adversarial Attacks

Aim

To implement a simple Generative Adversarial Network (GAN) that modifies fraudulent transactions so that a trained fraud detector fails to recognise them (an adversarial attack).

Software and Tools Required

- Google Colab (free, runs in the browser)
- Python libraries: NumPy, PyTorch, Matplotlib, Scikit-learn

Step 1: Import the Libraries

```
import numpy as np import torch import torch.nn as
nn import matplotlib.pyplot as plt from sklearn.datasets
import make_classification from sklearn.model_selection
import train_test_split from sklearn.preprocessing
import StandardScaler from sklearn.ensemble import
RandomForestClassifier torch.manual_seed(42)
np.random.seed(42)
```

Output: No output is shown. A green tick means the cell ran successfully.

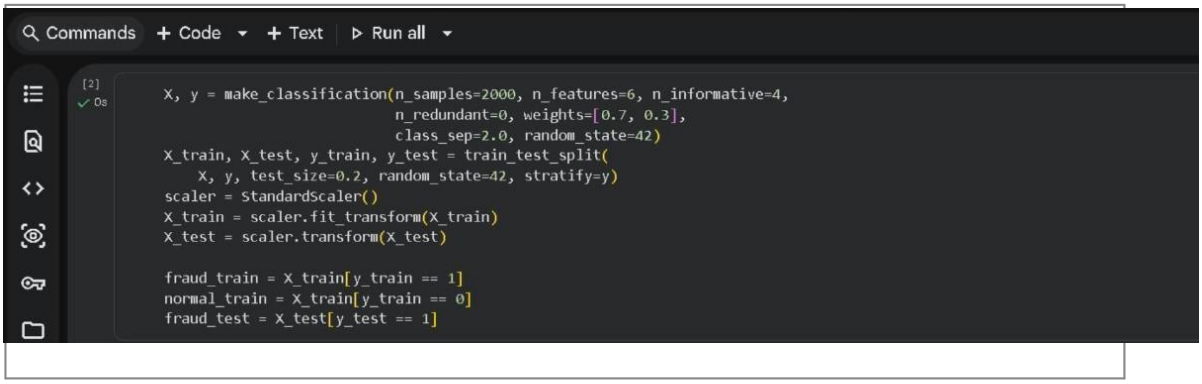


Step 2: Create and Prepare the Dataset

```
X, y = make_classification(n_samples=2000, n_features=6, n_informative=4,
n_redundant=0, weights=[0.7, 0.3],                              class_sep=2.0,
random_state=42) X_train, X_test, y_train, y_test = train_test_split(     X,
y, test_size=0.2, random_state=42, stratify=y) scaler = StandardScaler()
X_train = scaler.fit_transform(X_train) X_test
= scaler.transform(X_test)

fraud_train = X_train[y_train == 1]
normal_train = X_train[y_train == 0] fraud_test
= X_test[y_test == 1]
```

Output: No output is shown. A green tick means the cell ran successfully.



Step 3: Build the Target Fraud Detector

```
detector = RandomForestClassifier(n_estimators=100, random_state=42)
detector.fit(X_train, y_train) print("Detector accuracy:",
detector.score(X_test, y_test))
print("Fraud detected before attack:", detector.predict(fraud_test).mean())
```

Output obtained:

```
Detector accuracy: 0.9825
Fraud detected before attack: 0.9666666666666667
```



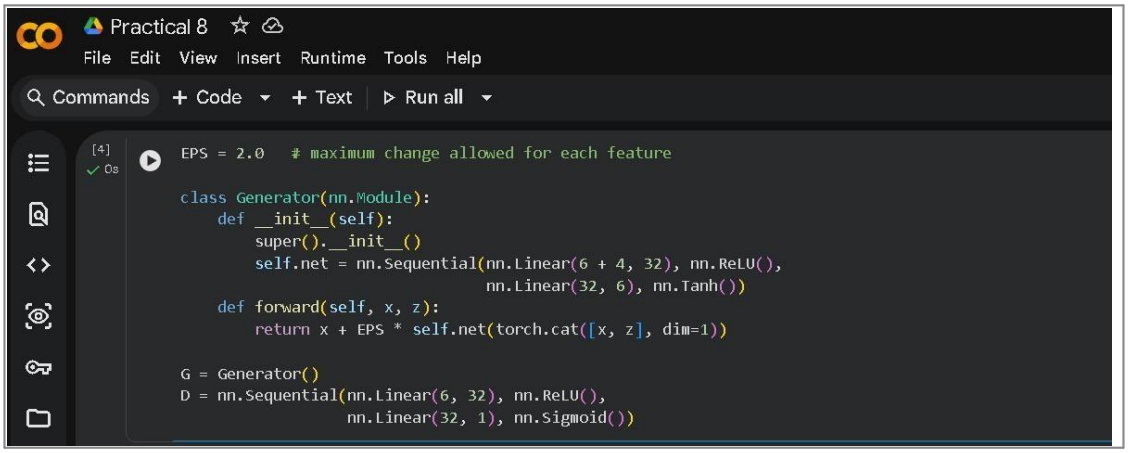
Step 4: Build the Generator and Discriminator

```
EPS = 2.0   # maximum change allowed for each feature

class Generator(nn.Module):     def __init__(self):
super().__init__()          self.net = nn.Sequential(nn.Linear(6 + 4,
32), nn.ReLU(),                                     nn.Linear(32,
6), nn.Tanh())      def forward(self, x, z):          return x + EPS
* self.net(torch.cat([x, z], dim=1))

G = Generator() D = nn.Sequential(nn.Linear(6, 32),
nn.ReLU(),                    nn.Linear(32,
1), nn.Sigmoid())
```

Output: No output is shown. A green tick means the cell ran successfully.



Step 5: Train the GAN

```
fraud_t = torch.tensor(fraud_train, dtype=torch.float32)

normal_t = torch.tensor(normal_train, dtype=torch.float32)

opt_G = torch.optim.Adam(G.parameters(), lr=0.001)
opt_D = torch.optim.Adam(D.parameters(), lr=0.001) loss_fn
= nn.BCELoss()
```

```
for epoch in range(3000):     x = fraud_t[torch.randint(0,
len(fraud_t), (64,))]     z = torch.rand(64, 4)     adv =
G(x, z)

    nrm = normal_t[torch.randint(0, len(normal_t), (64,))]
batch = torch.cat([adv.detach(), nrm])     labels =

torch.tensor(detector.predict(batch.numpy()),

dtype=torch.float32).view(-1, 1)     d_loss =

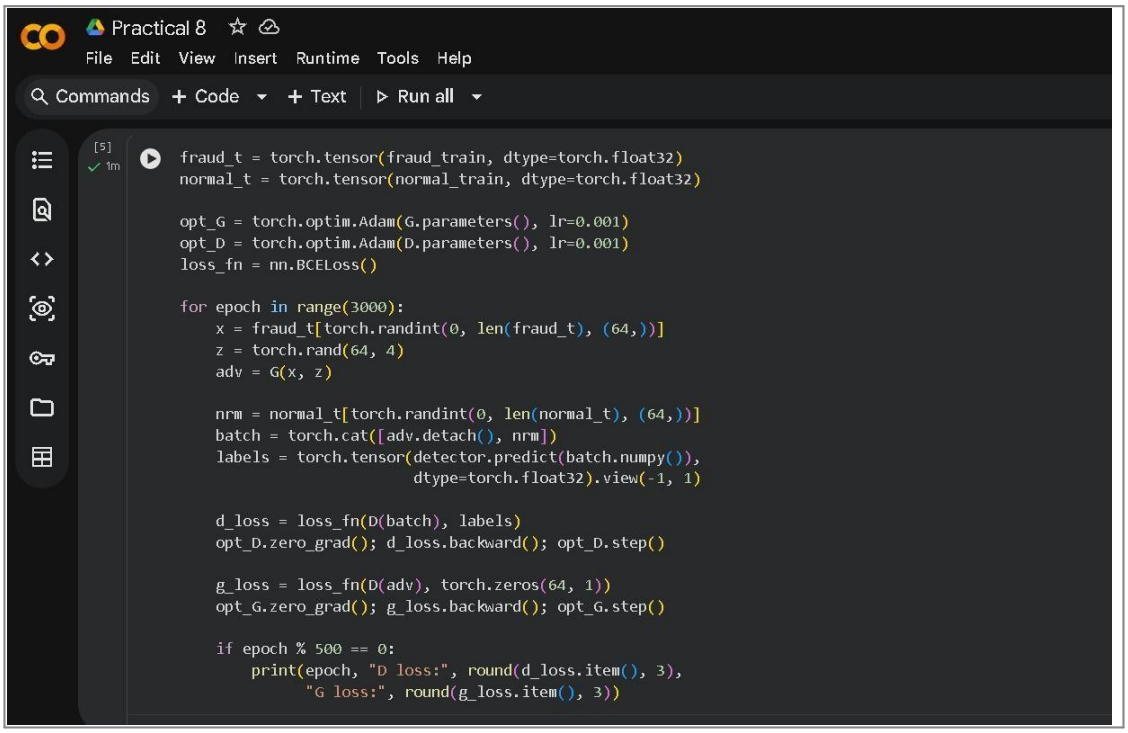
loss_fn(D(batch), labels)

opt_D.zero_grad(); d_loss.backward(); opt_D.step()

g_loss = loss_fn(D(adv), torch.zeros(64, 1))

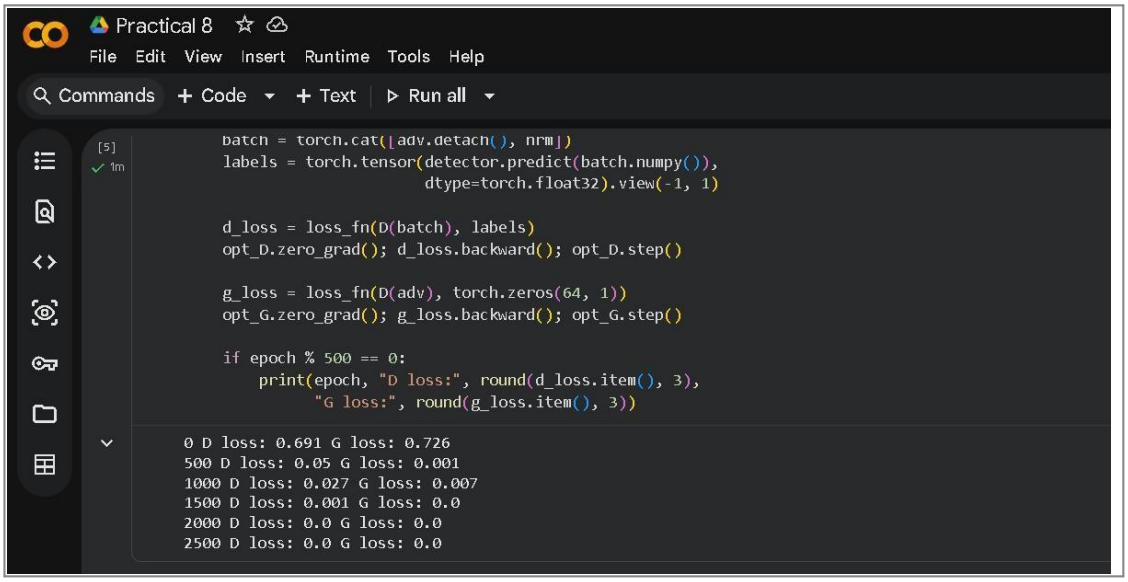
opt_G.zero_grad(); g_loss.backward(); opt_G.step()

    if epoch % 500 == 0:
        print(epoch, "D loss:", round(d_loss.item(), 3),               "G
loss:", round(g_loss.item(), 3))
```



Output obtained:

```
0 D loss: 0.691 G loss: 0.726
500 D loss: 0.05 G loss: 0.001
1000 D loss: 0.027 G loss: 0.007
1500 D loss: 0.001 G loss: 0.0
2000 D loss: 0.0 G loss: 0.0
2500 D loss: 0.0 G loss: 0.0
```



Step6: Test the Attack on Unseen Frauds

```
with torch.no_grad():
    fraud_test_t = torch.tensor(fraud_test, dtype=torch.float32)

    noise = torch.rand(len(fraud_test_t), 4)
    adv_test = G(fraud_test_t, noise).cpu().numpy()

before = detector.predict(fraud_test).mean()
after = detector.predict(adv_test).mean()

print("Fraud detected BEFORE attack:", round(before, 4))
print("Fraud detected AFTER attack :", round(after, 4))

print(
    "Average change per feature  :",
    round(np.abs(adv_test - fraud_test).mean(), 3)
)
```

Output obtained:

```
Fraud detected BEFORE attack: 0.9667
Fraud detected AFTER attack : 0.0
Average change per feature  : 1.861
```

```
[6]
with torch.no_grad():
    fraud_test_t = torch.tensor(fraud_test, dtype=torch.float32)
    adv_test = G(fraud_test_t, torch.rand(len(fraud_test_t), 4)).numpy()

before = detector.predict(fraud_test).mean()
after = detector.predict(adv_test).mean()
print("Fraud detected BEFORE attack:", round(before, 4))
print("Fraud detected AFTER attack :", round(after, 4))
print("Average change per feature  :", round(np.abs(adv_test - fraud_test).mean(), 3))

Fraud detected BEFORE attack: 0.9667
Fraud detected AFTER attack : 0.0
Average change per feature  : 1.861
```

Step 7: Draw the Result Graph

```
plt.bar(["Before attack", "After attack"], [before * 100, after * 100],
color=["green", "red"]) plt.ylabel("Fraud detected (%)") plt.title("Effect
of GAN adversarial attack on fraud detector") plt.show()
```

Output obtained: A bar chart. The green bar (before attack) is almost 97 and the red bar (after attack) is not visible because its value is 0%.



Conclusion

A simple GAN can learn to modify fraud transactions so that a detector wrongly calls them normal, even though the attacker never sees inside the detector. This shows that machine learning security systems can be attacked and that high accuracy on normal data does not guarantee safety against deliberate attacks. A common defence is **adversarial training**: the adversarial samples are added to the training data and the detector is trained again, so that it learns to recognise them.

PRACTICAL 9

Evaluating Detector Performance: Feature Engineering

Aim

To evaluate how feature engineering (creating new, more useful features from the raw data) improves the performance of a fraud detector.

Software and Tools Required

- Google Colab (free, runs in the browser)
- Python libraries: NumPy, Pandas, Matplotlib, Scikit-learn

Dataset Used

The program creates 5000 sample transactions. About 10% (525) are fraud and 90% (4475) are normal. Each transaction has 5 raw features. The data is artificial and is generated so that frauds behave differently from normal transactions.

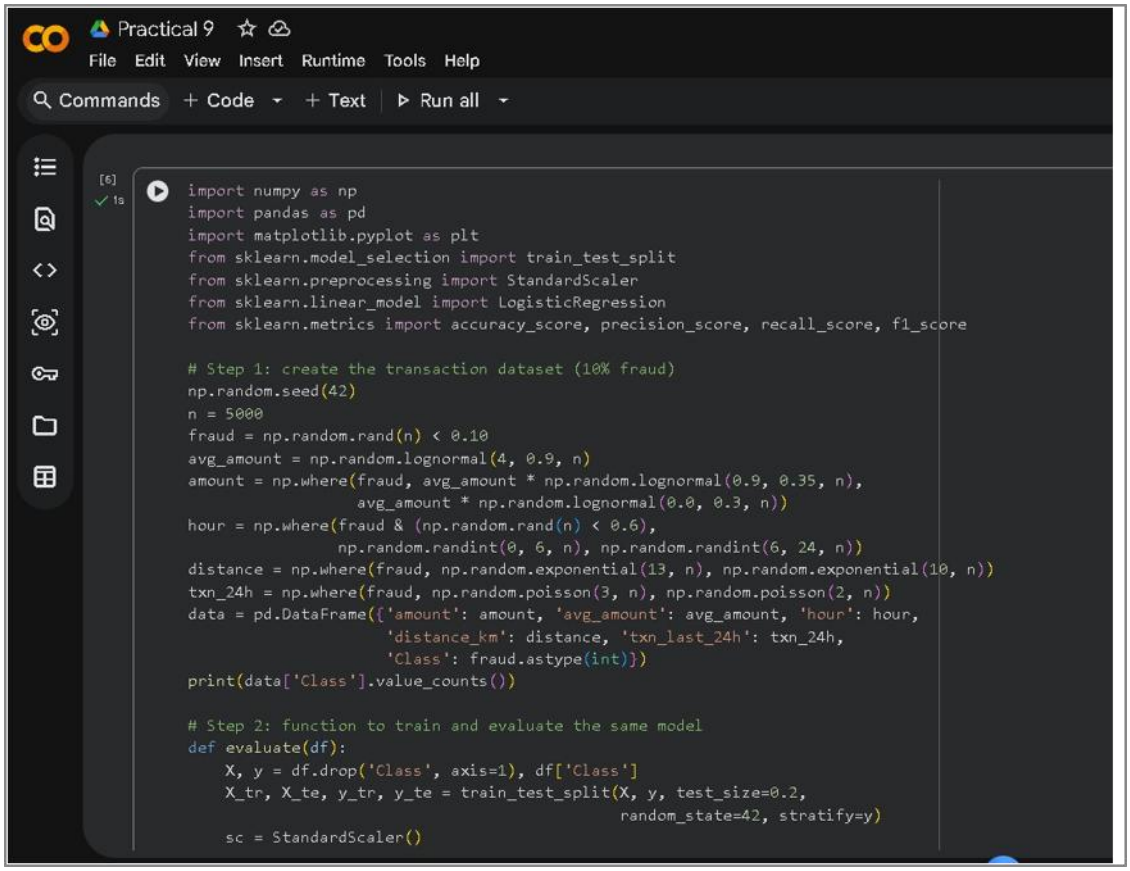
Column	Meaning	How fraud differs
amount	Amount of this transaction	About 2.5 times the customer's usual spending
avg_amount	What this customer usually spends	No difference (just the customer's habit)
hour	Hour of the day (0 to 23)	60% of frauds happen between 0 and 5 (night)
distance_km	Distance from the customer's home	Slightly farther (average 13 km against 10 km)
txn_last_24h	Transactions in the last 24 hours	Slightly more (average 3 against 2)
Class	0 = normal, 1 = fraud	(the answer to be predicted)

Complete Code (All Steps Together)

Paste this complete code into one Colab cell and run it. Every part is explained on the following pages.

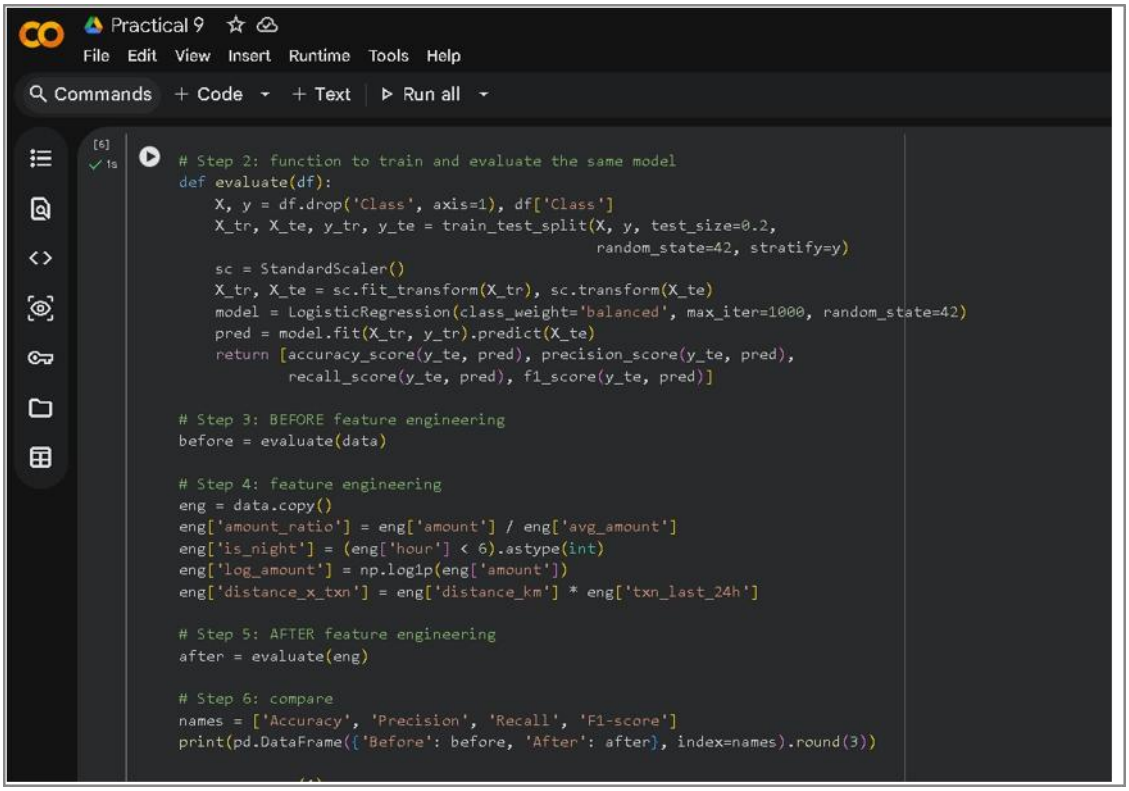
```
import numpy as np import pandas as pd import matplotlib.pyplot as plt from
sklearn.model_selection import train_test_split from sklearn.preprocessing import
StandardScaler from sklearn.linear_model import LogisticRegression from
sklearn.metrics import accuracy_score, precision_score, recall_score, f1_score

# Step 1: create the transaction dataset (10% fraud) np.random.seed(42) n = 5000 fraud
= np.random.rand(n) < 0.10 avg_amount = np.random.lognormal(4, 0.9, n) amount =
np.where(fraud, avg_amount * np.random.lognormal(0.9, 0.35, n),
avg_amount * np.random.lognormal(0.0, 0.3, n)) hour = np.where(fraud &
(np.random.rand(n) < 0.6),                np.random.randint(0, 6, n),
np.random.randint(6, 24, n)) distance = np.where(fraud, np.random.exponential(13, n),
np.random.exponential(10, n)) txn_24h = np.where(fraud, np.random.poisson(3, n),
np.random.poisson(2, n)) data = pd.DataFrame({'amount': amount, 'avg_amount':
avg_amount, 'hour': hour,                    'distance_km': distance,
'txn_last_24h': txn_24h,                    'Class': fraud.astype(int)})
print(data['Class'].value_counts())
```



```
# Step 2: function to train and evaluate the same model def evaluate(df):     X, y = df.drop('Class', axis=1), df['Class']     X_tr, X_te, y_tr, y_te = train_test_split(X, y, test_size=0.2,                                                 random_state=42, stratify=y)     sc = StandardScaler()     X_tr, X_te = sc.fit_transform(X_tr), sc.transform(X_te)     model = LogisticRegression(class_weight='balanced', max_iter=1000, random_state=42)     pred = model.fit(X_tr, y_tr).predict(X_te)     return [accuracy_score(y_te, pred), precision_score(y_te, pred), recall_score(y_te, pred), f1_score(y_te, pred)]

# Step 3: BEFORE feature engineering
before = evaluate(data)
```



```
# Step 4: feature engineering eng = data.copy()
eng['amount_ratio'] = eng['amount'] / eng['avg_amount']
eng['is_night'] = (eng['hour'] < 6).astype(int)
eng['log_amount'] = np.log1p(eng['amount'])
eng['distance_x_txn'] = eng['distance_km'] * eng['txn_last_24h']

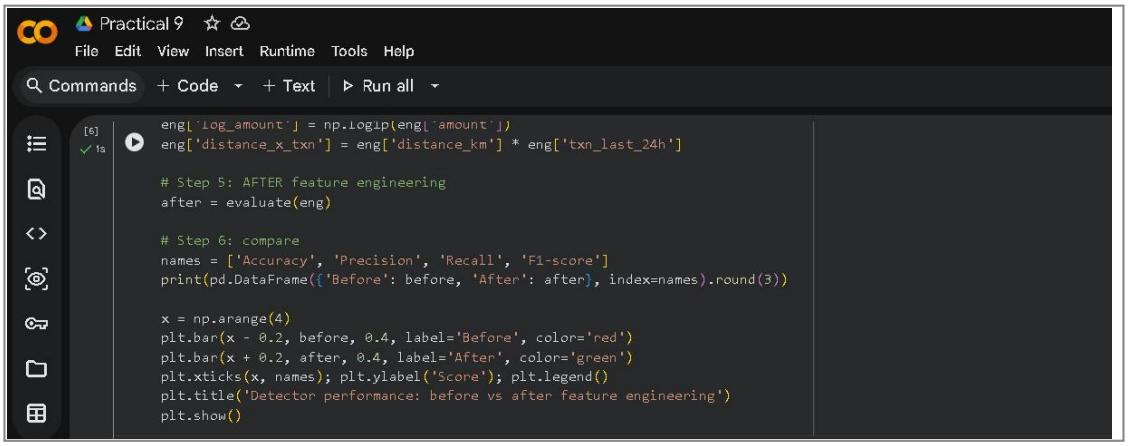
# Step 5: AFTER feature engineering
after = evaluate(eng)

# Step 6: compare names = ['Accuracy', 'Precision', 'Recall', 'F1-score']
print(pd.DataFrame({'Before': before, 'After': after}, index=names).round(3))

x = np.arange(4) plt.bar(x - 0.2, before, 0.4, label='Before',
color='red') plt.bar(x + 0.2, after, 0.4, label='After',
color='green') plt.xticks(x, names); plt.ylabel('Score'); plt.legend()
plt.title('Detector performance: before vs after feature engineering')
plt.show()
```

Complete Code as Run in Colab

The three screenshots below show the complete code in the notebook cell. The explanation of every part follows.



Step 0: Import the Libraries

```
import numpy as np import pandas as pd import
matplotlib.pyplot as plt from
sklearn.model_selection import train_test_split from
sklearn.preprocessing import StandardScaler from
sklearn.linear_model import LogisticRegression
from sklearn.metrics import accuracy_score, precision_score, recall_score, f1_score
```

Step 1: Create the Transaction Dataset

```
np.random.seed(42) n = 5000 fraud = np.random.rand(n) < 0.10 avg_amount =
np.random.lognormal(4, 0.9, n) amount = np.where(fraud, avg_amount *
np.random.lognormal(0.9, 0.35, n),                    avg_amount *
np.random.lognormal(0.0, 0.3, n)) hour = np.where(fraud & (np.random.rand(n) < 0.6),
np.random.randint(0, 6, n), np.random.randint(6, 24, n)) distance = np.where(fraud,
np.random.exponential(13, n), np.random.exponential(10, n)) txn_24h = np.where(fraud,
np.random.poisson(3, n), np.random.poisson(2, n)) data = pd.DataFrame({'amount':
amount, 'avg_amount': avg_amount, 'hour': hour,                    'distance_km':
distance, 'txn_last_24h': txn_24h,                    'Class': fraud.astype(int)})
print(data['Class'].value_counts())
```

Output obtained:

```
Class
0     4475
1     525
Name: count, dtype: int64
```

Step 2: Write a Function to Train and Test the Model

```
def evaluate(df):     X, y = df.drop('Class', axis=1), df['Class']     X_tr, X_te,
y_tr, y_te = train_test_split(X, y, test_size=0.2,
random_state=42, stratify=y)     sc = StandardScaler()     X_tr, X_te =
sc.fit_transform(X_tr), sc.transform(X_te)     model =
LogisticRegression(class_weight='balanced', max_iter=1000, random_state=42)     pred =
model.fit(X_tr, y_tr).predict(X_te)     return [accuracy_score(y_te, pred),
precision_score(y_te, pred),             recall_score(y_te, pred), f1_score(y_te,
pred)]
```

Step 3: Test the Model BEFORE Feature Engineering

```
before = evaluate(data)
```

Step 4: Feature Engineering (Create New Features)

```
eng = data.copy() eng['amount_ratio'] = eng['amount'] /
eng['avg_amount'] eng['is_night'] = (eng['hour'] <
6).astype(int) eng['log_amount'] =
np.log1p(eng['amount'])
eng['distance_x_txn'] = eng['distance_km'] * eng['txn_last_24h']
```

Step 5: Test the Model AFTER Feature Engineering

```
after = evaluate(eng)
```

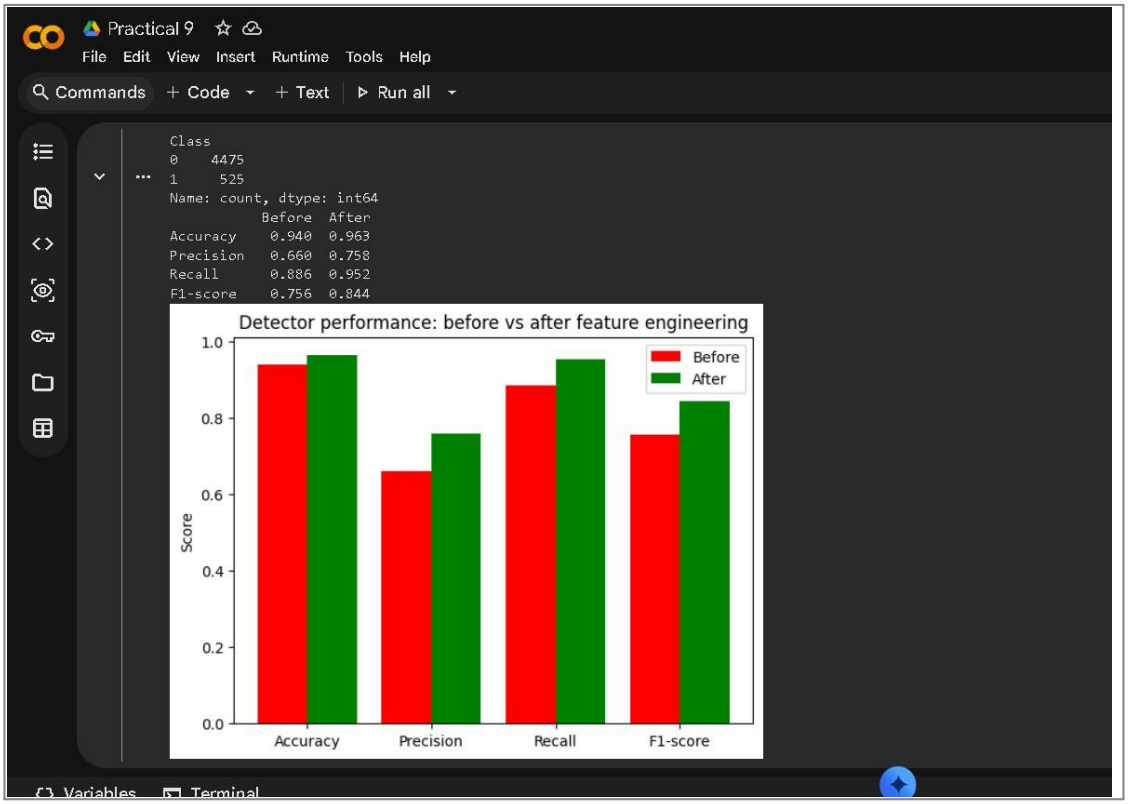
Step 6: Compare the Results and Draw the Graph

```
names = ['Accuracy', 'Precision', 'Recall', 'F1-score']
print(pd.DataFrame({'Before': before, 'After': after}, index=names).round(3))

x = np.arange(4) plt.bar(x - 0.2, before, 0.4, label='Before',
color='red') plt.bar(x + 0.2, after, 0.4, label='After',
color='green') plt.xticks(x, names); plt.ylabel('Score'); plt.legend()
plt.title('Detector performance: before vs after feature engineering')
plt.show()
```

Output obtained:

```
Class
0     4475
1     525
Name: count, dtype: int64
           Before  After
Accuracy    0.940  0.963
Precision   0.660  0.758
Recall      0.886  0.952
F1-score    0.756  0.844
```



Result

The same Logistic Regression detector was trained on the raw features and then on the raw features plus four engineered features. After feature engineering, accuracy rose from **0.940 to 0.963**, recall from **0.886 to 0.952** and the fraud F1-score from **0.756 to 0.844**.

Conclusion

Feature engineering improved the performance of the fraud detector without changing the model. Features such as amount_ratio and is_night express fraud patterns directly, so even a simple model can use them. Good features are often as important as the choice of model.

PRACTICAL 10

Evaluating Detector Performance: ROC and AUC

Aim

To evaluate fraud detectors using the ROC curve and the AUC score, and to compare Logistic Regression with Random Forest.

Software and Tools Required

- Google Colab (free, runs in the browser)
- Python libraries: NumPy, Pandas, Matplotlib, Scikit-learn

Dataset Used

The same 5000 sample transactions as in Practical 9 (about 10% fraud: 525 fraud and 4475 normal), with the 5 raw features and the 4 engineered features (amount_ratio, is_night, log_amount, distance_x_txn). Training uses 80% of the data (4000 rows) and testing uses 20% (1000 rows, with 105 frauds).

Procedure (Exact Steps)

1. Open Google Colab, click **File → New notebook** and rename it to **Practical 10**.
2. Paste the complete code into **one cell** and run it with **Shift + Enter**.
3. Take screenshots of the code, the AUC output and the ROC graph for the journal.

Complete Code as Run in Colab

The complete code is in one Colab cell, shown in the three screenshots below. Each part is explained on the following pages.

Step 1: Import the Libraries

```
import numpy as np import pandas as pd import
matplotlib.pyplot as plt from
sklearn.model_selection import train_test_split from
sklearn.preprocessing import StandardScaler from
sklearn.linear_model import LogisticRegression from
```

```
sklearn.ensemble import RandomForestClassifier from
sklearn.metrics import roc_curve, roc_auc_score
```

In easy words: We take out the tools we need. Nothing is printed.

Explanation:

- `numpy, pandas, matplotlib`: numbers, data tables and graphs.
- `train_test_split, StandardScaler`: split the data and bring all features to the same scale.
- `LogisticRegression, RandomForestClassifier`: the two detectors we compare.
- `roc_curve, roc_auc_score`: calculate the ROC points and the AUC.

Step 2: Create the Dataset

```
np.random.seed(42) n = 5000 fraud = np.random.rand(n) < 0.10 avg_amount =
np.random.lognormal(4, 0.9, n) amount = np.where(fraud, avg_amount *
np.random.lognormal(0.9, 0.35, n),                    avg_amount *
np.random.lognormal(0.0, 0.3, n)) hour = np.where(fraud & (np.random.rand(n) < 0.6),
np.random.randint(0, 6, n), np.random.randint(6, 24, n)) distance = np.where(fraud,
np.random.exponential(13, n), np.random.exponential(10, n)) txn_24h = np.where(fraud,
np.random.poisson(3, n), np.random.poisson(2, n)) data = pd.DataFrame({'amount':
amount, 'avg_amount': avg_amount, 'hour': hour,                       'distance_km':
distance, 'txn_last_24h': txn_24h}) data['amount_ratio'] = data['amount'] /
data['avg_amount'] data['is_night'] = (data['hour'] < 6).astype(int)
data['log_amount'] = np.log1p(data['amount']) data['distance_x_txn'] =
data['distance_km'] * data['txn_last_24h'] y = fraud.astype(int)
print("Fraud transactions:", y.sum(), "of", n)
```

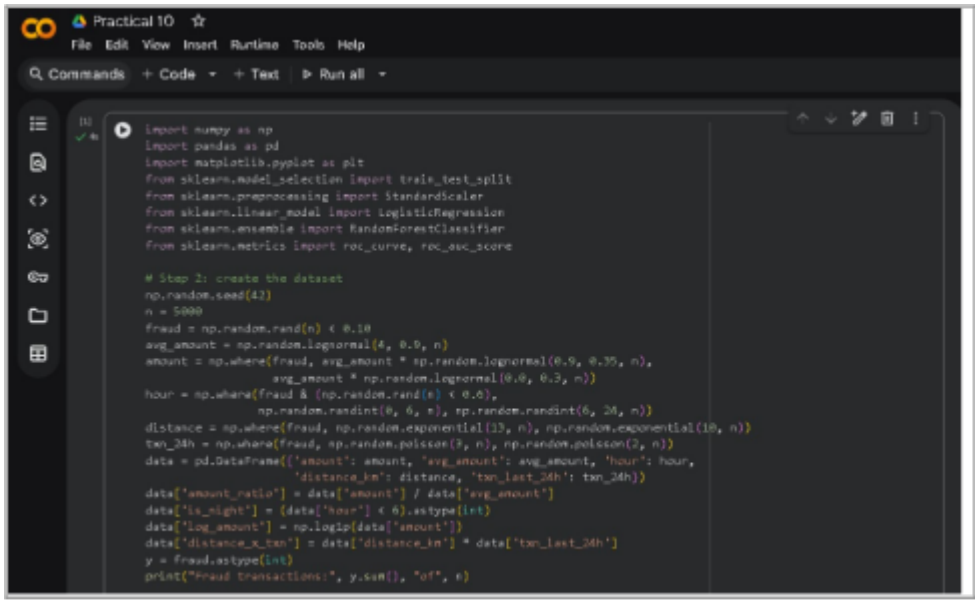
In easy words: We create 5000 sample transactions (about 10% fraud) as in Practical 9 and add the 4 engineered features. The answers (fraud or not) are kept separately in **y**.

Explanation:

- `fraud = np.random.rand(n) < 0.10`: about 10% of the transactions are marked as fraud.
- The next lines create the 5 raw features. Frauds get bigger amounts than usual, more night-time hours, slightly larger distances and more recent transactions.

- `data['amount_ratio'] ... data['distance_x_txn']`: the 4 engineered features from Practical 9.
- `y = fraud.astype(int)`: the answer column (1 = fraud, 0 = normal), kept **outside** `dat` so the model cannot see it as a feature.
- `y.sum()`: counts the frauds, because each fraud is a 1.

Output obtained:



Understanding the output: 525 of the 5000 transactions are fraud (about 10%).

Step 3: Split, Scale and Train Two Detectors

```
X_train, X_test, y_train, y_test = train_test_split(
data, y, test_size=0.2, random_state=42, stratify=y) sc
= StandardScaler() X_train = sc.fit_transform(X_train)
X_test = sc.transform(X_test)

lr = LogisticRegression(class_weight='balanced', max_iter=1000, random_state=42) rf =
RandomForestClassifier(n_estimators=100, class_weight='balanced', random_state=42)
lr.fit(X_train, y_train) rf.fit(X_train, y_train)
```

In easy words: We keep 20% of the data hidden for testing, scale the numbers, and teach two different detectors with the other 80%. Nothing is printed.

Explanation:

- `train_test_split(..., test_size=0.2, stratify=y)`: 80% for training and 20% for testing. `stratify=y` keeps the same fraud percentage in both parts.
- `sc.fit_transform(X_train)` learns the scale from the training data. `sc.transform(X_test)` uses the **same** scale for the test data, which must stay unseen.
- `LogisticRegression(class_weight='balanced')`: a simple model that adds up the features with weights. `balanced` gives extra importance to the rare fraud cases.
- `RandomForestClassifier(n_estimators=100)`: a forest of 100 decision trees that vote together.
- `lr.fit(...)` and `rf.fit(...)`: teach both models using the training data and its answers.
-

Step 4: Get Probabilities, ROC Points and AUC

```
lr_prob = lr.predict_proba(X_test)[:, 1]
rf_prob = rf.predict_proba(X_test)[:, 1]

lr_fpr, lr_tpr, _ = roc_curve(y_test, lr_prob)
rf_fpr, rf_tpr, _ = roc_curve(y_test, rf_prob) lr_auc
= roc_auc_score(y_test, lr_prob) rf_auc =
roc_auc_score(y_test, rf_prob) print("AUC - Logistic
Regression:", round(lr_auc, 3)) print("AUC - Random
Forest      :", round(rf_auc, 3))
```

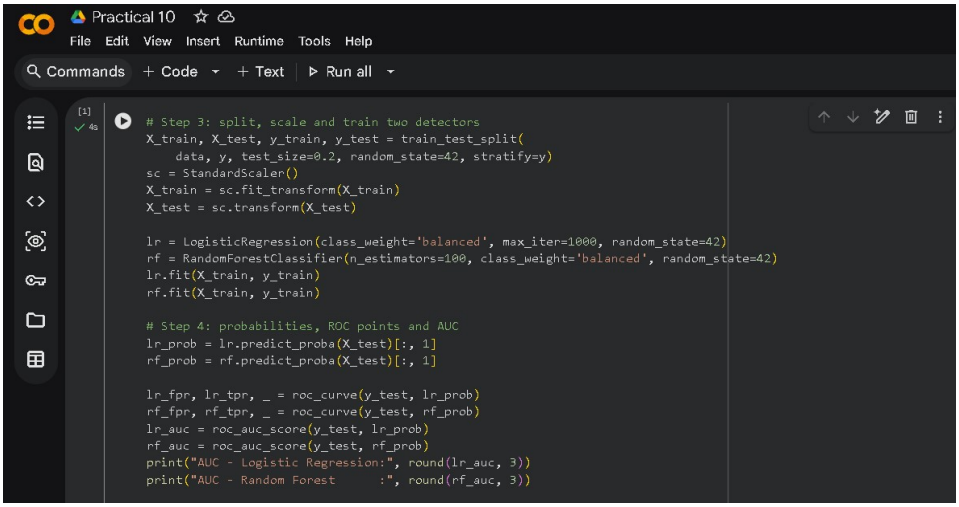
In easy words: Instead of only "fraud or not", we ask each model **how sure** it is. The computer then tries many thresholds, finds the frauds caught and false alarms for each, and measures the area under the resulting curve.

Explanation:

- `predict_proba(X_test)`: gives two columns, the probability of normal and the probability of fraud. `[:, 1]` keeps only the **fraud** column.
- `roc_curve(y_test, lr_prob)`: compares the real answers with the probabilities and returns the false positive rates, the true positive rates and the thresholds. We keep the first two, and the underscore _ ignores the thresholds.

- `roc_auc_score(y_test, lr_prob)`: calculates the AUC. The same is done for Random Forest.
- `round(lr_auc, 3)`: shows 3 decimals.

Output obtained:



Understanding the output:

- Logistic Regression: AUC = **0.990**. If you pick one random fraud and one random normal transaction, the model gives the fraud the higher probability about 99% of the time.
- Random Forest: AUC = **0.976**. Also excellent, but slightly lower.

Step 5: Draw the ROC Curves

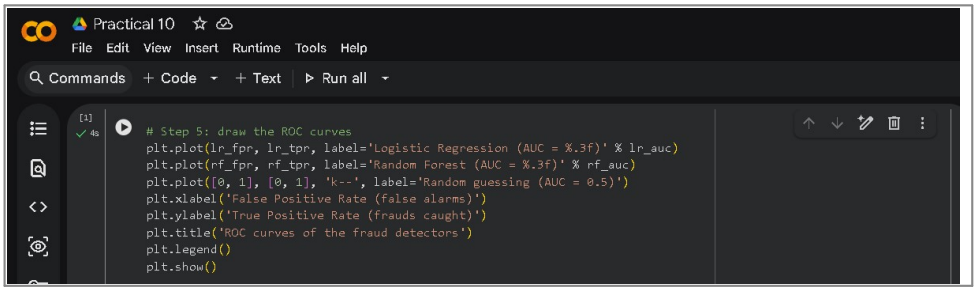
```
plt.plot(lr_fpr, lr_tpr, label='Logistic Regression (AUC = %.3f)' % lr_auc)
plt.plot(rf_fpr, rf_tpr, label='Random Forest (AUC = %.3f)' % rf_auc)
plt.plot([0, 1], [0, 1], 'k--', label='Random guessing (AUC = 0.5)')
plt.xlabel('False Positive Rate (false alarms)') plt.ylabel('True Positive
Rate (frauds caught)') plt.title('ROC curves of the fraud detectors')
plt.legend() plt.show()
```

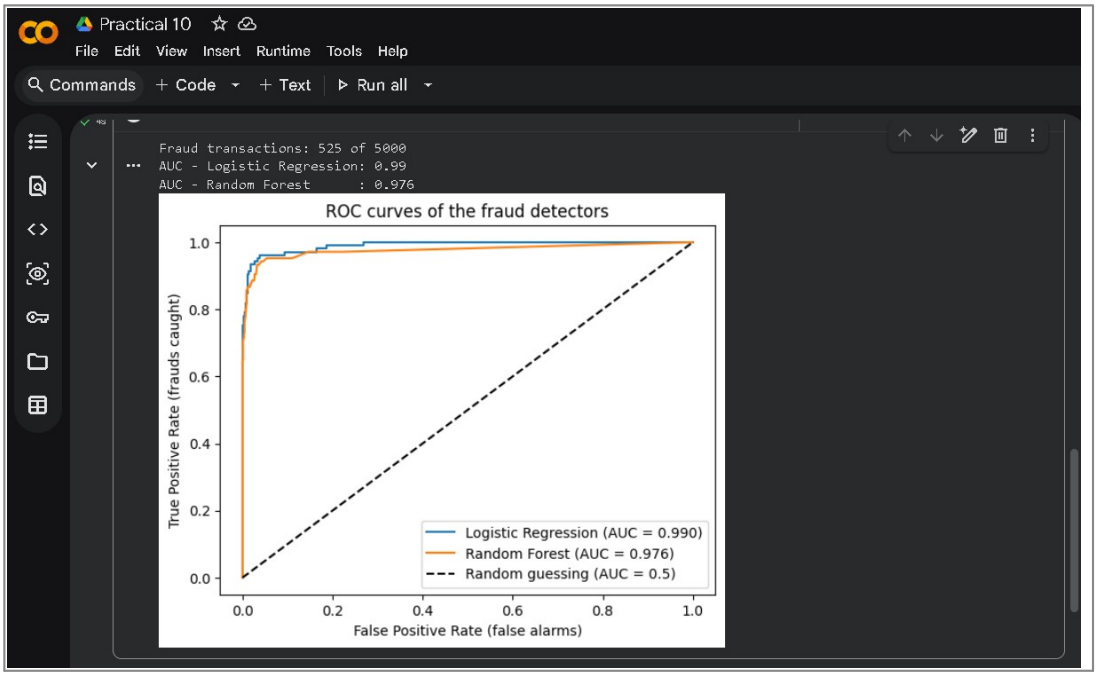
In easy words: One line for each detector. Every point on a line is one threshold: frauds caught (up) against false alarms (right). The dashed diagonal shows pure guessing.

Explanation:

- `plt.plot(lr_fpr, lr_tpr, label=...)`: draws the Logistic Regression curve. `%.3f` shows 3 decimals in the label. The Random Forest curve is drawn the same way.
- `plt.plot([0, 1], [0, 1], 'k--')`: the dashed diagonal line of a guessing detector (AUC = 0.5).
- `plt.xlabel, plt.ylabel, plt.title, plt.legend`: add the labels, the title and the names of the lines.
- `plt.show()`: displays the graph.

Output obtained: The AUC values and the ROC graph, as shown in Colab.





Screenshot 4: Output (AUC values and ROC graph)

How to read the graph:

- **Top-left corner** is best. Both curves rise almost straight up the left side and then run along the top, so both detectors are very good.

- **The dashed diagonal** is random guessing. The further a curve is above it, the better the detector.
- **The blue line** (Logistic Regression) stays slightly above the orange line (Random Forest), which matches its higher AUC.

Result

Logistic Regression reached an AUC of **0.990** and Random Forest reached **0.976**. Both ROC curves stay close to the top-left corner and far above the random-guessing line.

Conclusion

The ROC curve shows how a detector behaves at every threshold, and the AUC summarises it in one number. Both detectors perform excellently, and Logistic Regression is slightly better on this data. ROC and AUC give a fairer evaluation than accuracy alone, because they do not depend on one threshold and are not misled by the rarity of fraud.

PRACTICAL 11

Cross-Validation and Testing for Data Reliability

Aim

To perform cross-validation on fraud detectors and to test whether their results are reliable, instead of trusting a single train/test split.

Software and Tools Required

- Google Colab (free, runs in the browser)
- Python libraries: NumPy, Pandas, Matplotlib, Scikit-learn

Dataset Used

The same 5000 sample transactions as in Practicals 9 and 10 (about 10% fraud: 525 fraud and 4475 normal), with the 5 raw features and the 4 engineered features. In 5-fold cross-validation, each fold has about 1000 transactions (about 105 frauds).

Complete Code as Run in Colab

The complete code is in one Colab cell, shown in the three screenshots below. Each part is explained on the following pages.

Step 1: Import the Libraries

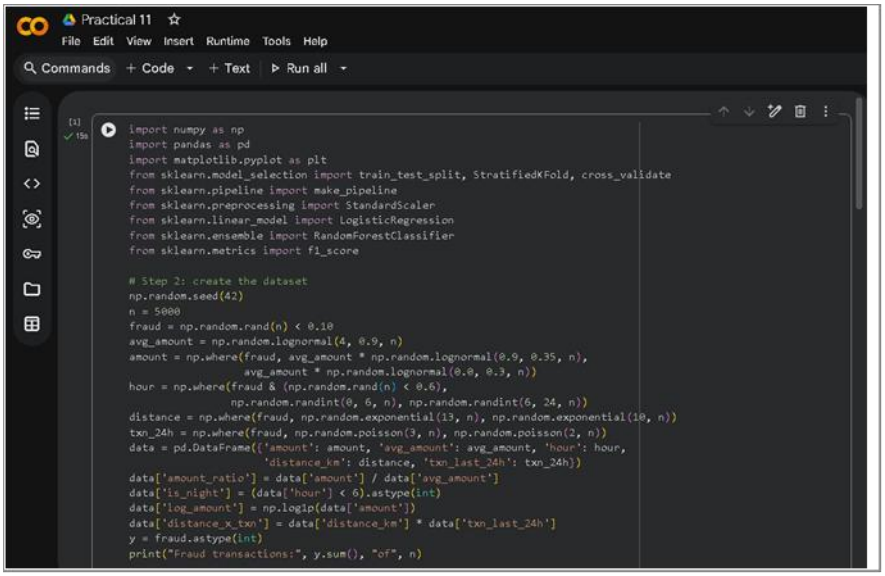
```
import numpy as np import pandas as pd import matplotlib.pyplot as plt from
sklearn.model_selection import train_test_split, StratifiedKFold, cross_validate from
sklearn.pipeline import make_pipeline from sklearn.preprocessing import
StandardScaler from sklearn.linear_model import LogisticRegression from
sklearn.ensemble import RandomForestClassifier from sklearn.metrics import f1_score
```

Step 2: Create the Dataset

```
np.random.seed(42) n = 5000 fraud = np.random.rand(n) < 0.10 avg_amount =
np.random.lognormal(4, 0.9, n) amount = np.where(fraud, avg_amount *
np.random.lognormal(0.9, 0.35, n),                    avg_amount *
np.random.lognormal(0.0, 0.3, n)) hour = np.where(fraud & (np.random.rand(n) < 0.6),
np.random.randint(0, 6, n), np.random.randint(6, 24, n)) distance = np.where(fraud,
np.random.exponential(13, n), np.random.exponential(10, n)) txn_24h = np.where(fraud,
np.random.poisson(3, n), np.random.poisson(2, n)) data = pd.DataFrame({'amount':
amount, 'avg_amount': avg_amount, 'hour': hour,                       'distance_km':
distance, 'txn_last_24h': txn_24h}) data['amount_ratio'] = data['amount'] /
data['avg_amount'] data['is_night'] = (data['hour'] < 6).astype(int)
data['log_amount'] = np.log1p(data['amount']) data['distance_x_txn'] =
data['distance_km'] * data['txn_last_24h'] y = fraud.astype(int)
print("Fraud transactions:", y.sum(), "of", n)
```

Output obtained:

```
Fraud transactions: 525 of 5000
```



Step 3: Is One Train/Test Split Reliable?

```
for seed in [0, 1, 2, 3, 4]:     X_tr, X_te, y_tr, y_te = train_test_split(
data, y, test_size=0.2, random_state=seed, stratify=y)     model =
make_pipeline(StandardScaler(),
LogisticRegression(class_weight='balanced', max_iter=1000))     model.fit(X_tr, y_tr)
    print("Split", seed, "-> F1-score:", round(f1_score(y_te, model.predict(X_te)), 3))
```

Output obtained:

```
Split 0 -> F1-score: 0.848
Split 1 -> F1-score: 0.867
Split 2 -> F1-score: 0.869
Split 3 -> F1-score: 0.84
Split 4 -> F1-score: 0.855
```

Step 4: 5-Fold Cross-Validation of Two Detectors

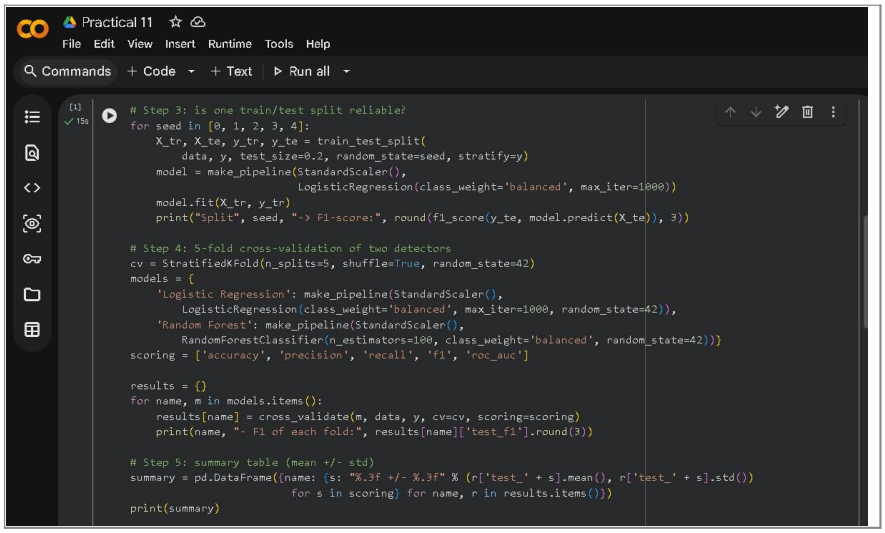
```
cv = StratifiedKFold(n_splits=5, shuffle=True, random_state=42)
models = {     'Logistic Regression':
make_pipeline(StandardScaler(),
        LogisticRegression(class_weight='balanced', max_iter=1000, random_state=42)),
```

```
    'Random Forest': make_pipeline(StandardScaler(),
RandomForestClassifier(n_estimators=100, class_weight='balanced', random_state=42))} scoring
= ['accuracy', 'precision', 'recall', 'f1', 'roc_auc']

results = {} for name, m in
models.items():
    results[name] = cross_validate(m, data, y, cv=cv, scoring=scoring)
print(name, "- F1 of each fold:", results[name]['test_f1'].round(3))
```

Output obtained:

```
Logistic Regression - F1 of each fold: [0.868 0.842 0.87  0.852 0.855]
Random Forest - F1 of each fold: [0.911 0.888 0.879 0.913 0.891]
```



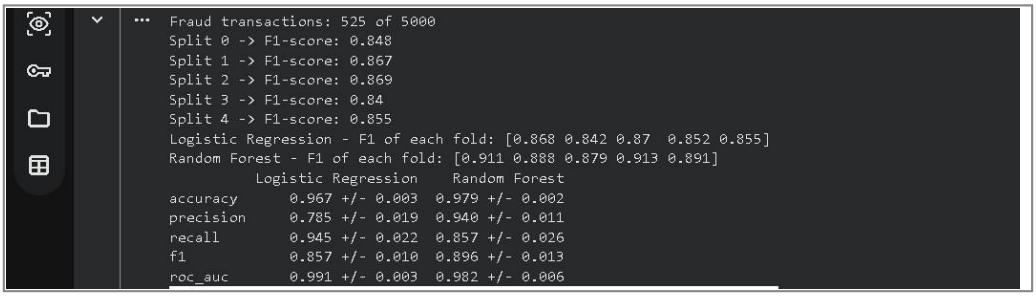
Step 5: Summary Table (Mean and Spread)

```
summary = pd.DataFrame({name: {s: "%.3f +/- %.3f" % (r['test_' + s].mean(), r['test_' + s].std()
for s in scoring} for name, r in results.items()}) print(summary)
```

Output obtained:

```
           Logistic Regression    Random Forest
accuracy       0.967 +/- 0.003  0.979 +/- 0.002
precision      0.785 +/- 0.019  0.940 +/- 0.011
recall         0.945 +/- 0.022  0.857 +/- 0.026
f1             0.857 +/- 0.010  0.896 +/- 0.013
roc_auc        0.991 +/- 0.003  0.982 +/- 0.006
```

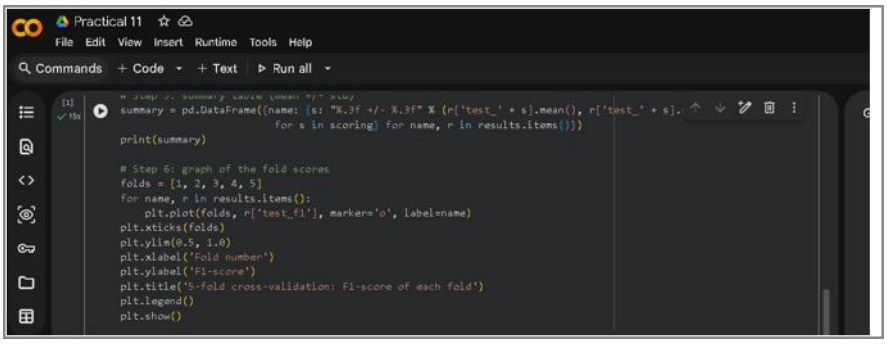
Colab output of Steps 2 to 5:



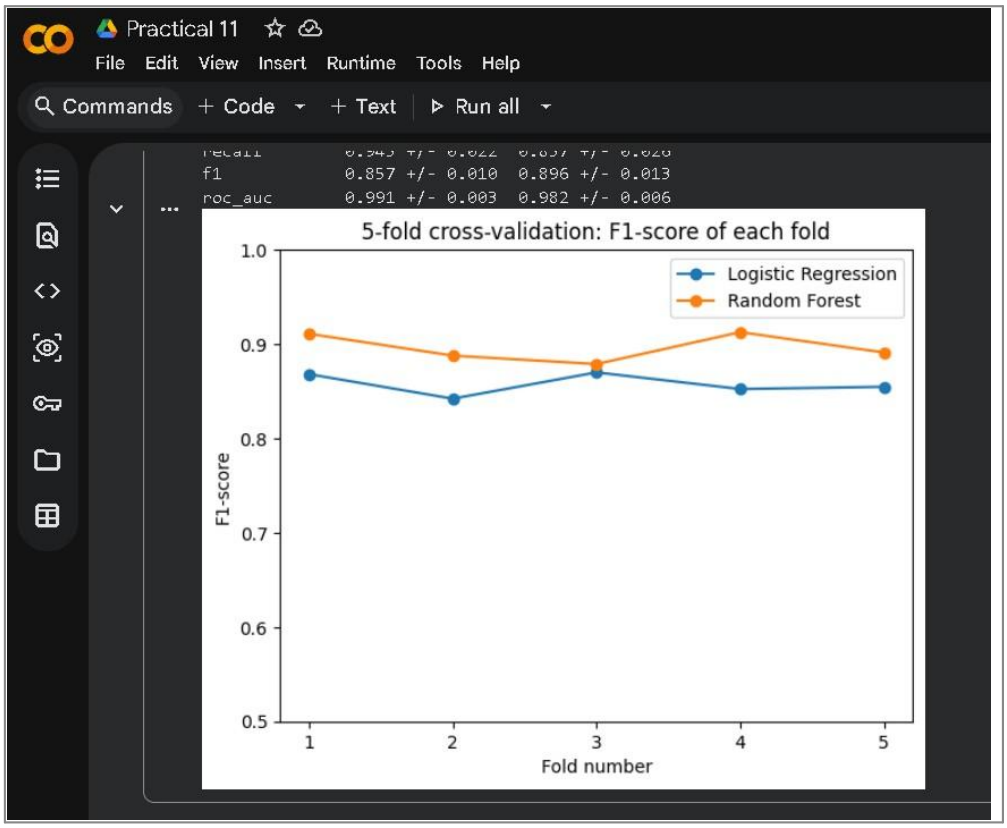
```
Fraud transactions: 525 of 5000
Split 0 -> F1-score: 0.848
Split 1 -> F1-score: 0.867
Split 2 -> F1-score: 0.869
Split 3 -> F1-score: 0.84
Split 4 -> F1-score: 0.855
Logistic Regression - F1 of each fold: [0.868 0.842 0.87  0.852 0.855]
Random Forest - F1 of each fold: [0.911 0.888 0.879 0.913 0.891]
           Logistic Regression    Random Forest
accuracy      0.967 +/- 0.003  0.979 +/- 0.002
precision     0.785 +/- 0.019  0.940 +/- 0.011
recall        0.945 +/- 0.022  0.857 +/- 0.026
f1            0.857 +/- 0.010  0.896 +/- 0.013
roc_auc       0.991 +/- 0.003  0.982 +/- 0.006
```

Step 6: Draw the Graph of the Fold Scores

```
folds = [1, 2, 3, 4, 5] for
name, r in results.items():
    plt.plot(folds, r['test_f1'], marker='o', label=name)
plt.xticks(folds) plt.ylim(0.5, 1.0) plt.xlabel('Fold
number') plt.ylabel('F1-score') plt.title('5-fold cross-
validation: F1-score of each fold') plt.legend() plt.show()
```



Output obtained: The graph, as shown in Colab.



Result

Five different train/test splits gave F1-scores from 0.840 to 0.869, which shows that one split is not fully reliable. In 5-fold cross-validation the scores were stable (standard deviation about 0.01). Random Forest reached a mean F1-score of **0.896** and Logistic Regression **0.857**. Logistic Regression had the higher AUC (**0.991** against 0.982) and recall (0.945 against 0.857).

Conclusion

Cross-validation gives a more trustworthy estimate of a detector's performance than a single split, because every row is used for testing once and the result is an average with a measure of its spread. The small standard deviations show that the results of both detectors are reliable. Random Forest is better when precision and F1-score matter, and Logistic Regression is better when catching as many frauds as possible matters.